RentaMovil Software Design Documentation

Nicole Dayana Ramirez Vargas

Thiago Fabian Rojas Guevara

David Santiago Erez Rodriguez

National Training Service (SENA)

Software Analysis and Development - Course 3145556

October 15, 2026

Abstract

This document describes the RentaMovil design and relates its interfaces, requirements, microservices architecture, documented patterns, and relational model in SQL Server. It presents the rental process, the current bank-transfer payment with receipt and administrative approval, the security decisions, and the data dictionary. It is addressed to instructors, evaluators, and the development team.

Keywords: RentaMovil, software design, microservices, hexagonal architecture, interfaces, SQL Server, data model.

Table of contents

- Abstract
- 1. Introduction
- 1.1 Purpose, audience, and scope
- 1.2 Objectives
- 1.3 Conceptual framework
- 1.4 Elaboration methodology
- 2. Actors and requirements
- 2.1 Requirements and business rules
- 2.2 Non-functional requirements
- 3. Graphical user interfaces
- 4. Architectural model
- 4.1 Technologies
- 4.2 Implementation status
- 5. Design patterns
- 6. Data model
- 6.1 Data protection
- 7. Security, quality, and operations
- 8. Conclusions
- References
- Appendix A. Data dictionary
- Appendix B. Graphical user interfaces
- Appendix C. Document inventory
- Appendix D. Support diagrams

1. Introduction

RentaMovil digitizes the rental cycle: availability search, booking, transfer and review, fleet administration, physical rental, invoicing, notifications, and GPS. This document consolidates source artifacts and does not replace them (RentaMovil, 2026, 01-context/overview.md; 03-product/vision.md).

1.1 Purpose, audience, and scope

Aimed at instructors, evaluators, and the development team. It covers the client web and mobile applications and the administrative web: authentication, catalog, bookings, insurance, transfer, invoices, fleet, branches, maintenance, rental execution, GPS, notifications, and roles. An automatic payment gateway and a mobile administrative application are listed as future evolution (RentaMovil, 2026, 01-context/scope.md; 03-product/vision.md).

1.2 Objectives

General objective: to document the RentaMovil design with traceability among interfaces, architecture, patterns, and data model. Specific objectives: to describe actors and requirements; to explain the applications and services; to summarize the SQL Server entities; to record the decisions relevant to the evaluation.

1.3 Conceptual framework

The design uses bounded contexts, deployable microservices, hexagonal architecture, REST APIs/events, and a normalized relational model. The interface applies role-based navigation and a shared visual system (RentaMovil, 2026, 02-domain/domain-map.md; 05-architecture/hexagonal-architecture.md; 06-data/normalization-assessment.md).

1.4 Elaboration methodology

Context, scope, requirements, architecture, ADRs, models, catalog, and UX artifacts were reviewed; these were contrasted with the decisions of the team and synthesized with internal citations. The team's corrections regarding SQL Server, transfers, and the immutable date prevail;

contradictory states are flagged. No executable code was audited and no production measurements were taken.

2. Actors and requirements

Table 1.
Actors and their responsibilities

Actor	Responsibilities
Client	Searches availability, books, pays by transfer, and consults bookings, invoices, and notifications.
Administrator	Manages fleet, branches, and maintenance; reviews payments; records pick-up and return; consults authorized GPS data.
Superadministrator	Inherits administrative permissions; manages users, roles, and bank accounts.

2.1 Requirements and business rules

The backlog contains 31 user stories and seven epics: identity, fleet, booking, rental execution, payment/invoicing, GPS, and notifications (RentaMovil, 2026, 04-requirements/user-stories.md).

Table 2.
Business rules

Rule	Description
Booking	A reservation links client, vehicle, dates, branches, and at most one optional insurance.
Return	By the team's instruction, the return date cannot be changed after the booking is created. This prevails over earlier versions of the design, which allowed extensions.
Transfer	The client selects an account, records the amount/reference, and uploads a receipt; the administrator approves or rejects it, with date and reason.
Confirmation	The booking is confirmed after the payment is approved.
Privacy	Do not include real values of passwords, identifiers, tokens, or banking data.

2.2 Non-functional requirements

The sources cover performance, availability, scalability, security, observability, maintainability, portability, and disaster recovery. Numerical goals must come from the NFR matrix; no verified measurements are declared. It includes JWT, permissions, auditing, secret

protection, secure transport, and idempotency (RentaMovil, 2026, 04-requirements/non-functional.md; 00-governance/security-policy.md).

3. Graphical user interfaces

The approved visual manual defines the brand, palette, Poppins/Inter, hierarchy, spacing, components, states, icons, forms, and loading. Interfaces are grouped by audience: public, client, administrator, and superadministrator. Navigation contains 21 documented routes; there are views shared by role (RentaMovil Team, 2026, Identidad_visual.pdf; RentaMovil, 2026, 12-ux-ui/navigation-map.md; screen-descriptions.md).

Table 3.
Screens by interface group

Group	Screens
Public	Home; catalog /vehicles; vehicle detail /vehicles/:id; sign-in, sign-up, and account recovery /auth/*.
Client	Profile, bookings, booking wizard, payment, booking detail, and notifications inbox.
Administration	Fleet, vehicle creation and editing, branches, bookings, insurance, and tracking.
Superadministration	Users and roles, and bank accounts.

The catalog requires a date or a branch; the actions in the detail view depend on permissions. The wizard gathers vehicle and date selection, insurance, terms, and the transfer process. Loading, error, empty, and confirmation states follow the visual identity.

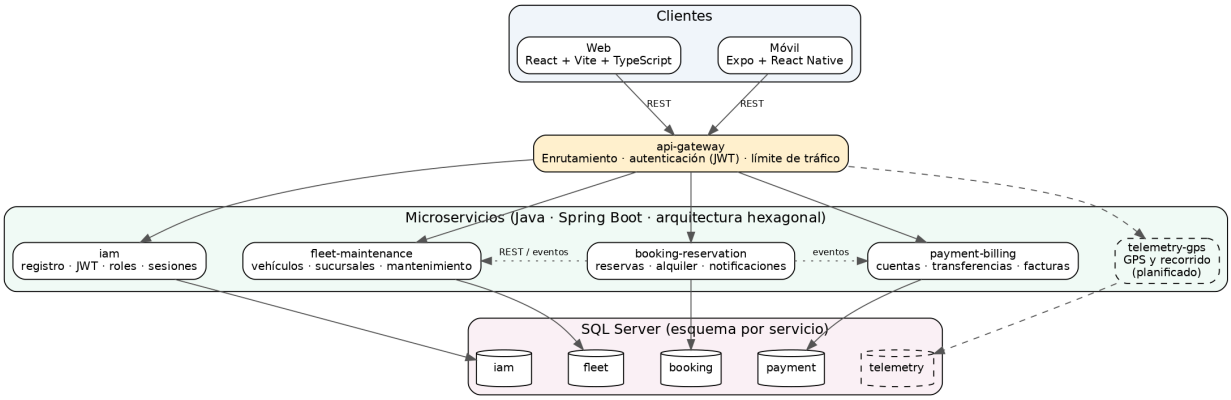
4. Architectural model

Client-server architecture: React/Vite web and Expo/React Native mobile consume REST through an API Gateway. Backend in Java/Spring Boot organized as deployable microservices; synchronous REST and asynchronous events. Seven logical contexts are grouped into five physical services plus the gateway; booking-reservation includes rental execution and notifications. A shared SQL Server database uses schemas and per-service write ownership (RentaMovil, 2026, 05-architecture/overview.md; 09-microservices/service-catalog.md; 06-data/README.md).

Table 4.
Services and responsibilities

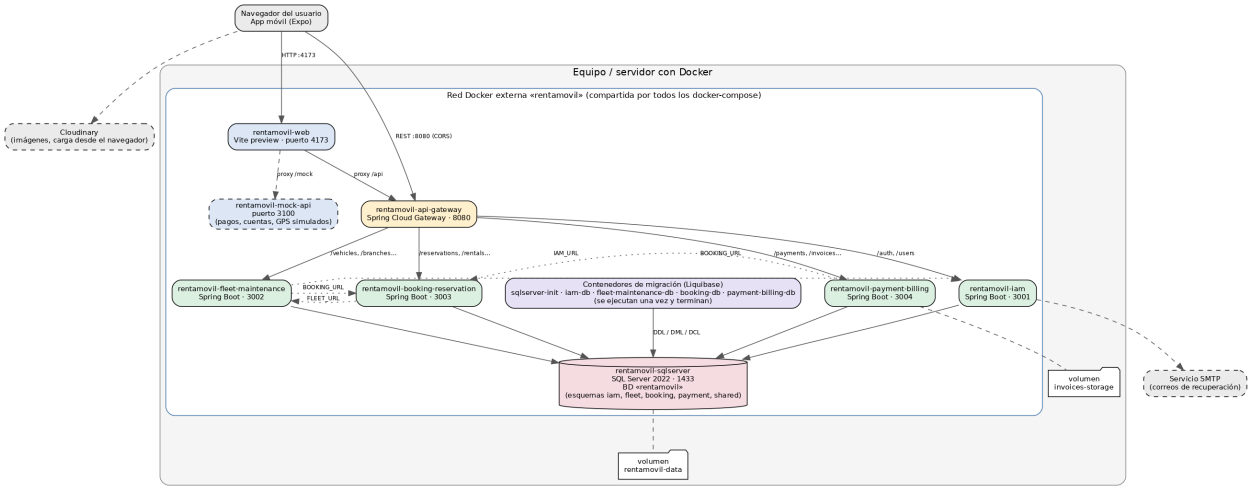
Service	Responsibility
api-gateway	Routing, authentication, traffic limiting.
iam	Sign-up, JWT, roles, permissions, sessions.
fleet-maintenance	Vehicles, branches, availability, and maintenance.
booking-reservation	Bookings, physical rental, and notifications.
payment-billing	Accounts, transfers, review, and invoices.
telemetry-gps	GPS devices and route tracking.

Figure 24. *RentaMovil microservices architecture*



Source: prepared by the RentaMovil team, 2026, from the project repositories. The dotted lines indicate communication between services; the telemetry-gps service appears as planned.

Figure 25. *Docker deployment diagram (containers, ports, and network)*



Source: prepared by the RentaMovil team, 2026, from the code and the docker-compose files of the project repositories. The dotted lines between services correspond to the IAM_URL, FLEET_URL, and BOOKING_URL variables; the mock-api container simulates functionality without a real service.

4.1 Technologies

Table 5.
Technologies by layer

Layer	Technology
Web	React, Vite, and JavaScript (JSX), with some modules in TypeScript; modules by functionality, routes, and components.
Mobile	Expo, React Native, TypeScript, Expo Router, Zustand.
Backend	Java, Spring Boot, hexagonal architecture.
Data	Microsoft SQL Server, schemas/migrations per service.
Infrastructure	Docker and environments described by DevOps.

4.2 Implementation status

As of October 15, 2026, the team's repositories contain code and automated tests for five of the six services in the catalog: api-gateway, iam, fleet-maintenance, booking-reservation, and payment-billing. Each service with a database has its migration repository (iam-db, fleet-maintenance-db, booking-db, and payment-billing-db). The web and mobile clients are in the rtm-frontend repository. The telemetry-gps service still has no repository and remains planned.

This description is based on the structure of the repositories and does not constitute a functional audit of the code. The planning sources (service catalog and backlog) were prepared earlier and their state does not match the current one.

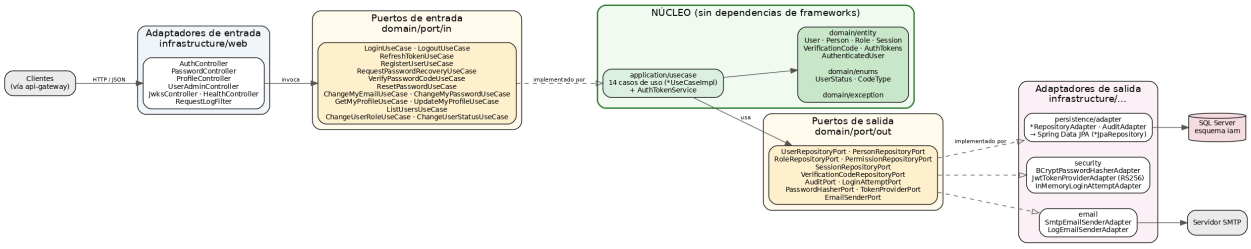
In the web application, some features (bank accounts, branches, insurance, and bookings on certain screens) still consume simulated data from the mock-api container shown in Figure 25; they must not be understood as integrated with the real backend.

5. Design patterns

Table 6.
Design patterns and their status

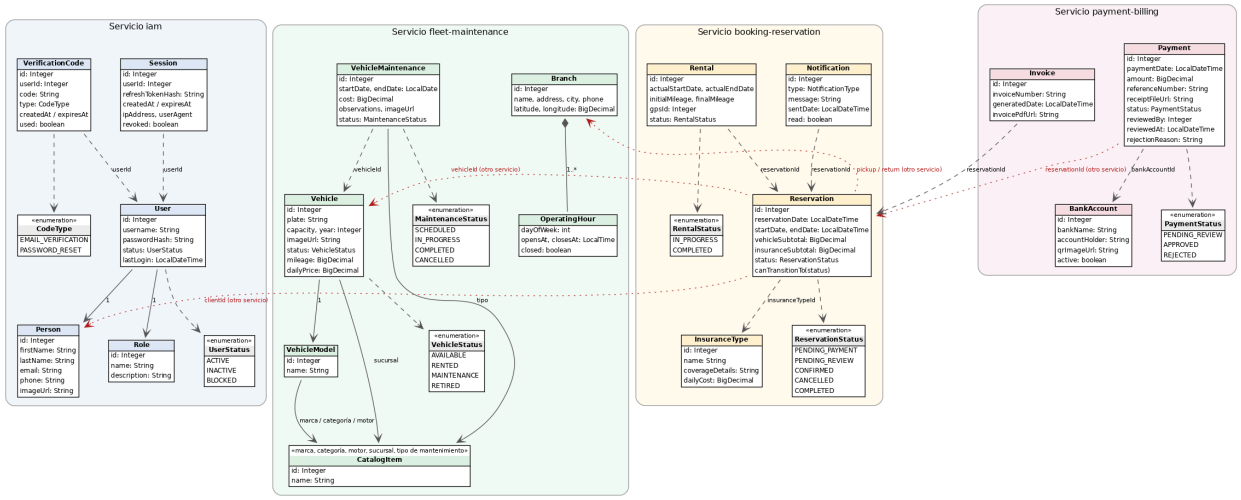
Pattern	Application	Status
Hexagonal	Domain isolated through ports/adapters.	Adopted in the backend.
Feature-based	Client modules by feature/domain.	Documented.
Component-driven	Reusable UI components.	Documented for mobile.
API Gateway	External entry point to services.	Gateway listed as active.
Schema per service	Table ownership per schema in a shared database.	ADR-009.
Domain events	Changes across domains and idempotency.	Events defined; transport subject to configuration.
Saga/Outbox/Circuit breaker	Distributed consistency and resilience.	To evaluate; implementation not asserted.

Figure 26. *Hexagonal architecture of the iam service (domain, ports, and adapters)*



Source: prepared by the RentaMovil team, 2026, from the package structure of the rtm-iam repository. The other services follow the same organization (domain, application, infrastructure).

Figure 27. *Class diagram of the domain per service*



Source: prepared by the RentaMovil team, 2026, from the classes in domain/entity and domain/enums of each service. The dotted red lines are identifier references between services, not foreign keys.

No Factory, Strategy, Observer, or CQRS patterns are attributed because the sources do not confirm their use. A proposed pattern is not the same as an implemented one (RentaMovil, 2026, 05-architecture/overview.md; hexagonal-architecture.md).

6. Data model

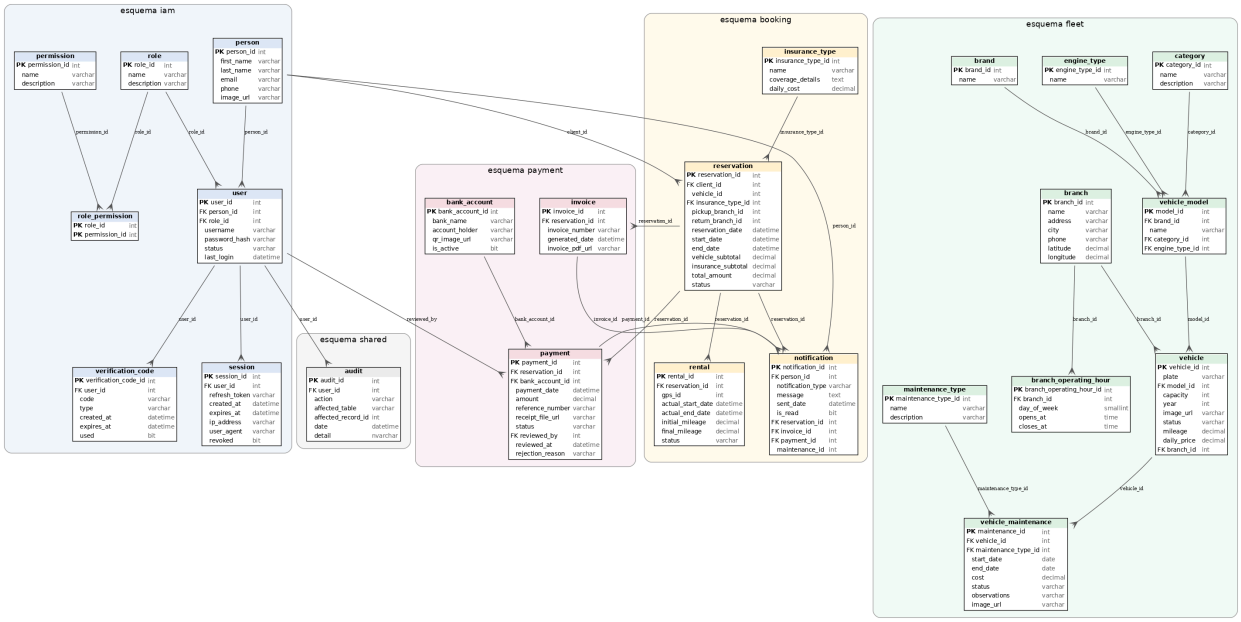
The physical model contains 26 domain tables in seven contexts plus a cross-cutting audit table. Conventions: snake_case, integer IDENTITY primary keys, relational foreign keys, and CHECK constraints for enumerated states. Schemas: iam, fleet, booking, payment, telemetry, and shared. Appendix A transcribes the source dictionary (RentaMovil, 2026, 06-data/models.md; data-dictionary.md; normalization-assessment.md).

The versioned migrations in the database repositories implement 24 tables (iam 7, fleet 9, booking 4, payment 3, and shared 1); the two remaining tables (gps and location) belong to the telemetry schema, which has no migrations yet.

Table 7.
Conceptual model by context

Context	Conceptual model
iam	person-user 1:1; user-role N:1; role-permission N:M; sessions and verification codes 1:N per user.
fleet	branch operating hours; brand-model-vehicle; vehicle-maintenance.
booking	the reservation relates person, vehicle, branches, and optional insurance; reservation-rental 1:1.
payment	bank account-payment 1:N; reservation-payment attempts 1:N; invoice-reservation 1:1.
telemetry	GPS-location 1:N; the rental assigns a GPS.
shared	audit: records the traceability of actions.

Figure 28. *Implemented physical data model (iam, fleet, booking, payment, and shared schemas)*



Source: prepared by the RentaMovil team, 2026, generated from the Liquibase migrations of the rtm-iam-db, rtm-fleet-maintenance-db, rtm-booking-db, and rtm-payment-billing-db repositories. Only the foreign keys declared within each database are drawn; the relationships between schemas (for example, reservation-vehicle or reservation-branch) are resolved in the service layer.

6.1 Data protection

password_hash is a bcrypt hash; a password is never displayed. Only technical field names appear; no real values of IDs, secrets, people, or banking data. Receipts and invoices are

storage references, not attachments (RentaMovil, 2026, 06-data/models.md; 00-governance/security-rules.md).

7. Security, quality, and operations

The documentation describes JWT RS256, RBAC, permissions, validation, auditing, secure secrets, and protected transport. Quality covers tests per layer and traceability; DevOps, containers and environments; operations, observability and incidents. With no executable results, design is reported, not metrics or certifications (RentaMovil, 2026, security-policy.md; 11-quality/testing-strategy.md; 10-devops/README.md; 13-operations/observability.md).

8. Conclusions

The design integrates approved interfaces, microservices, a reviewable transfer flow, and a shared SQL Server database. The team's decisions regarding the database, manual approval, and the immutable date are adopted. The telemetry-gps service remains future work, and the planning sources must be updated with the defined return date.

References

RentaMovil. (2026). 01-context/overview.md and scope.md [Internal documentation]. RentaMovil documentation repository.

RentaMovil. (2026). 02-domain/domain-map.md and entities-and-rules.md [Domain model]. RentaMovil documentation repository.

RentaMovil. (2026). 03-product/vision.md [Product vision]. RentaMovil documentation repository.

RentaMovil. (2026). 04-requirements/user-stories.md and non-functional.md [Requirements]. RentaMovil documentation repository.

RentaMovil. (2026). 05-architecture/overview.md and hexagonal-architecture.md [Architecture]. RentaMovil documentation repository.

RentaMovil. (2026). ADR-009-schema-per-service-and-migration-repos.md [Architecture decision record]. RentaMovil documentation repository.

RentaMovil. (2026). 06-data/models.md and data-dictionary.md [Data model and data dictionary]. RentaMovil documentation repository.

RentaMovil. (2026). 09-microservices/service-catalog.md [Service catalog]. RentaMovil documentation repository.

RentaMovil. (2026). 12-ux-ui/navigation-map.md and screen-descriptions.md [Interfaces]. RentaMovil documentation repository.

RentaMovil Team. (2026). Identidad_visual.pdf [Approved visual identity manual]. Design document folder.

RentaMovil. (2026). 00-governance/security-policy.md and security-rules.md [Security policies]. RentaMovil documentation repository.

RentaMovil. (2026). 10-devops/README.md; 11-quality/testing-strategy.md; 13-operations/observability.md [Operations and quality]. RentaMovil documentation repository.

RentaMovil. (2026). 12-ux-ui/design-system.md [Visual system]. RentaMovil documentation repository.

Appendix A. Data dictionary

Fields and constraints transcribed from the data model and updated with the migrations in the database repositories; no record data is included.

person — Identity and access

Table 8.
Fields of the person table

Field	Type	Nullable	Description	Constraints
person_id	int	No	Unique identifier	PK, IDENTITY
first_name	varchar(100)	No		
last_name	varchar(100)	No		
email	varchar(150)	No		Unique (`uq_person_email`): enforced by the database and not only by the application; two simultaneous records with the same email cannot both succeed
phone	varchar(20)	Yes		
image_url	varchar(500)	Yes	Profile photo URL; the image is stored in Cloudinary and only the link is kept here	

user — Identity and access

Table 9.
Fields of the user table

Field	Type	Nullable	Description	Constraints
user_id	int	No		PK, IDENTITY
person_id	int	No	1:1 relationship with `person`	FK → `person.person_id`, Unique
role_id	int	No	Direct foreign key; each user has exactly one role (not N:M)	FK → `role.role_id`
username	varchar(50)	No		Unique
password_hash	varchar(255)	No	bcrypt hash, cost ≥ 12	

Field	Type	Nullable	Description	Constraints
status	varchar(20)	No	Account status: `ACTIVE`, `INACTIVE`, or `BLOCKED`	`CHECK`
last_login	datetime	Yes		

role — Identity and access

Table 10.
Fields of the role table

Field	Type	Nullable	Description	Constraints
role_id	int	No		PK, IDENTITY
name	varchar(50)	No	`CLIENT`, `ADMIN`, or `SUPER_ADMIN`	Unique
description	varchar(255)	Yes		

permission — Identity and access

Table 11.
Fields of the permission table

Field	Type	Nullable	Description	Constraints
permission_id	int	No		PK, IDENTITY
name	varchar(50)	No	For example, `reservations:create`	Unique
description	varchar(255)	Yes		

role_permission — Identity and access

Table 12.
Fields of the role_permission table

Field	Type	Nullable	Description	Constraints
role_id	int	No		FK → `role.role_id`, part of the composite primary key
permission_id	int	No		FK → `permission.permission_id`, part of the composite primary key

verification_code — Identity and access

Table 13.
Fields of the verification_code table

Field	Type	Nullable	Description	Constraints
verification_code_id	int	No		PK, IDENTITY
user_id	int	No		FK → `user.user_id`
code	varchar(10)	No		
type	varchar(30)	No	For example, `EMAIL_VERIFICATION` and `PASSWORD_RESET`	
created_at	datetime	No		
expires_at	datetime	No		
used	boolean	No		

session — Identity and access

Table 14.
Fields of the session table

Field	Type	Nullable	Description	Constraints
session_id	int	No		PK, IDENTITY
user_id	int	No		FK → `user.user_id`
refresh_token	varchar(255)	No	See `00-governance/security-rules.md` for the rotation policy	
created_at	datetime	No		
expires_at	datetime	No	7 days from creation	
ip_address	varchar(45)	Yes		
user_agent	varchar(255)	Yes		
revoked	boolean	No		

branch — Fleet and maintenance

Table 15.
Fields of the branch table

Field	Type	Nullable	Description	Constraints
branch_id	int	No		PK, IDENTITY
name	varchar(100)	No		
address	varchar(200)	No		
city	varchar(100)	No		
phone	varchar(20)	Yes		
latitude	decimal(9,6)	Yes	Optional; defined together with `longitude` (both or neither)	`CHECK` between -90 and 90
longitude	decimal(9,6)	Yes	Optional; defined together with `latitude`	`CHECK` between -180 and 180

branch_operating_hour — Fleet and maintenance

Table 16.
Fields of the branch_operating_hour table

Field	Type	Nullable	Description	Constraints
branch_operating_hour_id	int	No		PK, IDENTITY
branch_id	int	No		FK → `branch.branch_id`
day_of_week	smallint	No	1–7 (Monday to Sunday)	`CHECK (day_of_week BETWEEN 1 AND 7)`
opens_at	time	No		
closes_at	time	No		

brand — Fleet and maintenance

Table 17.
Fields of the brand table

Field	Type	Nullable	Description	Constraints
brand_id	int	No		PK, IDENTITY
name	varchar(50)	No		Unique

vehicle_model — Fleet and maintenance

Table 18.
Fields of the vehicle_model table

Field	Type	Nullable	Description	Constraints
model_id	int	No		PK, IDENTITY
brand_id	int	No		FK → `brand.brand_id`
name	varchar(50)	No		
category_id	int	No		FK → `category.category_id`
engine_type_id	int	No		FK → `engine_type.engine_type_id`

category — Fleet and maintenance

Table 19.
Fields of the category table

Field	Type	Nullable	Description	Constraints
category_id	int	No	For example, economy, SUV, or luxury	PK, IDENTITY
name	varchar(50)	No		
description	varchar(255)	Yes		

engine_type — Fleet and maintenance

Table 20.
Fields of the engine_type table

Field	Type	Nullable	Description	Constraints
engine_type_id	int	No	For example, gasoline, diesel, electric, or hybrid	PK, IDENTITY
name	varchar(50)	No		

vehicle — Fleet and maintenance

Table 21.
Fields of the vehicle table

Field	Type	Nullable	Description	Constraints
vehicle_id	int	No		PK, IDENTITY
plate	varchar(10)	No		Unique

Field	Type	Nullable	Description	Constraints
model_id	int	No	Category and engine type are looked up through the model; they are not stored here	FK → `vehicle_model.model_id`
capacity	int	Yes	Number of passengers	
year	int	Yes		
image_url	varchar(255)	Yes		
status	varchar(30)	No	`AVAILABLE`, `RENTED`, `MAINTENANCE`, or `RETIRED`	`CHECK`
mileage	decimal(10,2)	No		
daily_price	decimal(10,2)	No		
branch_id	int	No	Vehicle's current branch	FK → `branch.branch_id`

vehicle_maintenance — Fleet and maintenance

Table 22.
Fields of the vehicle_maintenance table

Field	Type	Nullable	Description	Constraints
maintenance_id	int	No		PK, IDENTITY
vehicle_id	int	No		FK → `vehicle.vehicle_id`
maintenance_type_id	int	No		FK → `maintenance_type.maintenance_type_id`
start_date	date	No		
end_date	date	Yes	Null while in progress	
cost	decimal(10,2)	Yes		
observations	varchar(500)	Yes	Maintenance notes	
image_url	varchar(255)	Yes	URL of the image associated with the maintenance record	
status	varchar(30)	No	`SCHEDULED`, `IN_PROGRESS`, `COMPLETED`, or `CANCELLED` (cancelling replaces deleting the record)	`CHECK`

maintenance_type — Fleet and maintenance

Table 23.
Fields of the maintenance_type table

Field	Type	Nullable	Description	Constraints
maintenance_type_id	int	No	For example, oil change, tires, or inspection	PK, IDENTITY
name	varchar(50)	No		
description	varchar(255)	Yes		

insurance_type — Bookings

Table 24.
Fields of the insurance_type table

Field	Type	Nullable	Description	Constraints
insurance_type_id	int	No		PK, IDENTITY
name	varchar(100)	No		
coverage_details	text	Yes		
daily_cost	decimal(10,2)	No		

reservation — Bookings

Table 25.
Fields of the reservation table

Field	Type	Nullable	Description	Constraints
reservation_id	int	No		PK, IDENTITY
client_id	int	No		FK → `person.person_id`
vehicle_id	int	No		FK → `vehicle.vehicle_id`
insurance_type_id	int	Yes	Insurance is optional	FK → `insurance_type.insurance_type_id`
pickup_branch_id	int	No		FK → `branch.branch_id`
return_branch_id	int	No	The return branch can be changed up to 3 days before `end_date`	FK → `branch.branch_id`

Field	Type	Nullable	Description	Constraints
			(INV-013); it does not modify the return date	
reservation_date	datetime	No	Booking creation date	
start_date	datetime	No		
end_date	datetime	No		
vehicle_subtotal	decimal(10,2)	No		
insurance_subtotal	decimal(10,2)	No	0 if no insurance is selected	
total_amount	decimal(10,2)	No		
status	varchar(30)	No	`PENDING_PAYMENT`, `PENDING_REVIEW`, `CONFIRMED`, `CANCELLED`, or `COMPLETED`	`CHECK`

rental — Rental execution

Table 26.
Fields of the rental table

Field	Type	Nullable	Description	Constraints
rental_id	int	No		PK, IDENTITY
reservation_id	int	No	1:1 relationship with `reservation`; created when an administrator records the pick-up (RF4.1)	FK → `reservation.reservation_id`, Unique
gps_id	int	No		FK → `gps.gps_id`
actual_start_date	datetime	No		
actual_end_date	datetime	Yes	Null while `status = IN_PROGRESS`	
initial_mileage	decimal(10,2)	No		
final_mileage	decimal(10,2)	Yes	Set when the administrator records the return (RF4.2)	
status	varchar(30)	No	`IN_PROGRESS` or `COMPLETED`	`CHECK`

bank_account — Payments and invoicing

Table 27.
Fields of the bank_account table

Field	Type	Nullable	Description	Constraints
bank_account_id	int	No		PK, IDENTITY
bank_name	varchar(100)	No	For example, Bancolombia or Nequi	
account_holder	varchar(150)	No	Account holder name (the company)	
qr_image_url	varchar(255)	No	QR image shown to the client during payment; S3/MinIO storage	
is_active	boolean	No	Inactive accounts are hidden during payment, but keep their history	

payment — Payments and invoicing

Table 28.
Fields of the payment table

Field	Type	Nullable	Description	Constraints
payment_id	int	No		PK, IDENTITY
reservation_id	int	No		FK → `reservation.reservation_id`
bank_account_id	int	No	Bank account the client transferred to	FK → `bank_account.bank_account_id`
payment_date	datetime	No	Date on which the client registered the transfer	
amount	decimal(10,2)	No	Must be exactly equal to `reservation.total_amount`; see INV-001	
reference_number	varchar(100)	Yes	Transaction reference provided by the client	
receipt_file_url	varchar(255)	No	Receipt file uploaded by the client; S3/MinIO storage	
status	varchar(30)	No	`PENDING_REVIEW`, `APPROVED`, or `REJECTED`	`CHECK`

Field	Type	Nullable	Description	Constraints
reviewed_by	int	Yes	Administrator who reviewed it; null while `status = PENDING_REVIEW`	FK → `user.user_id`
reviewed_at	datetime	Yes	Null while `status = PENDING_REVIEW`	
rejection_reason	varchar(255)	Yes	Administrator's note; set only when `status = REJECTED`	

invoice — Payments and invoicing

Table 29.
Fields of the invoice table

Field	Type	Nullable	Description	Constraints
invoice_id	int	No		PK, IDENTITY
reservation_id	int	No	1:1 relationship with `reservation`	FK → `reservation.reservation_id`, Unique
invoice_number	varchar(50)	No		Unique
generated_date	datetime	No		
invoice_pdf_url	varchar(255)	Yes	PDF generated once when the invoice is created and saved to file storage (S3/MinIO); it is not regenerated on every query	

gps — Telemetry and GPS

Table 30.
Fields of the gps table

Field	Type	Nullable	Description	Constraints
gps_id	int	No	Physical device installed per vehicle when the rental starts	PK, IDENTITY
serial	varchar(100)	No		Unique
model	varchar(50)	Yes		
is_active	boolean	No		

location — Telemetry and GPS

Table 31.
Fields of the location table

Field	Type	Nullable	Description	Constraints
location_id	int	No		PK, IDENTITY
gps_id	int	No		FK → `gps.gps_id`
latitude	decimal(10,8)	No		
longitude	decimal(11,8)	No		
timestamp	datetime	No		

notification — Notifications

Table 32.
Fields of the notification table

Field	Type	Nullable	Description	Constraints
notification_id	int	No		PK, IDENTITY
person_id	int	No	Recipient	FK → `person.person_id`
notification_type	varchar(50)	No	For example, `RESERVATION_CREATED`, `PAYMENT_APPROVED`, and `PAYMENT_REJECTED`; one for each domain event that triggers a notification	`CHECK`
message	text	No		
sent_date	datetime	No		
is_read	boolean	No		
reservation_id	int	Yes	Set when the notification corresponds to a reservation	FK → `reservation.reservation_id`
invoice_id	int	Yes	Set when the notification corresponds to an invoice	FK → `invoice.invoice_id`
payment_id	int	Yes	Set when the notification corresponds to a payment	FK → `payment.payment_id`
maintenance_id	int	Yes	Set when the notification corresponds to a maintenance record	FK → `vehicle_maintenance.maintenance_id`

audit — Cross-cutting (audit)

Table 33.
Fields of the audit table

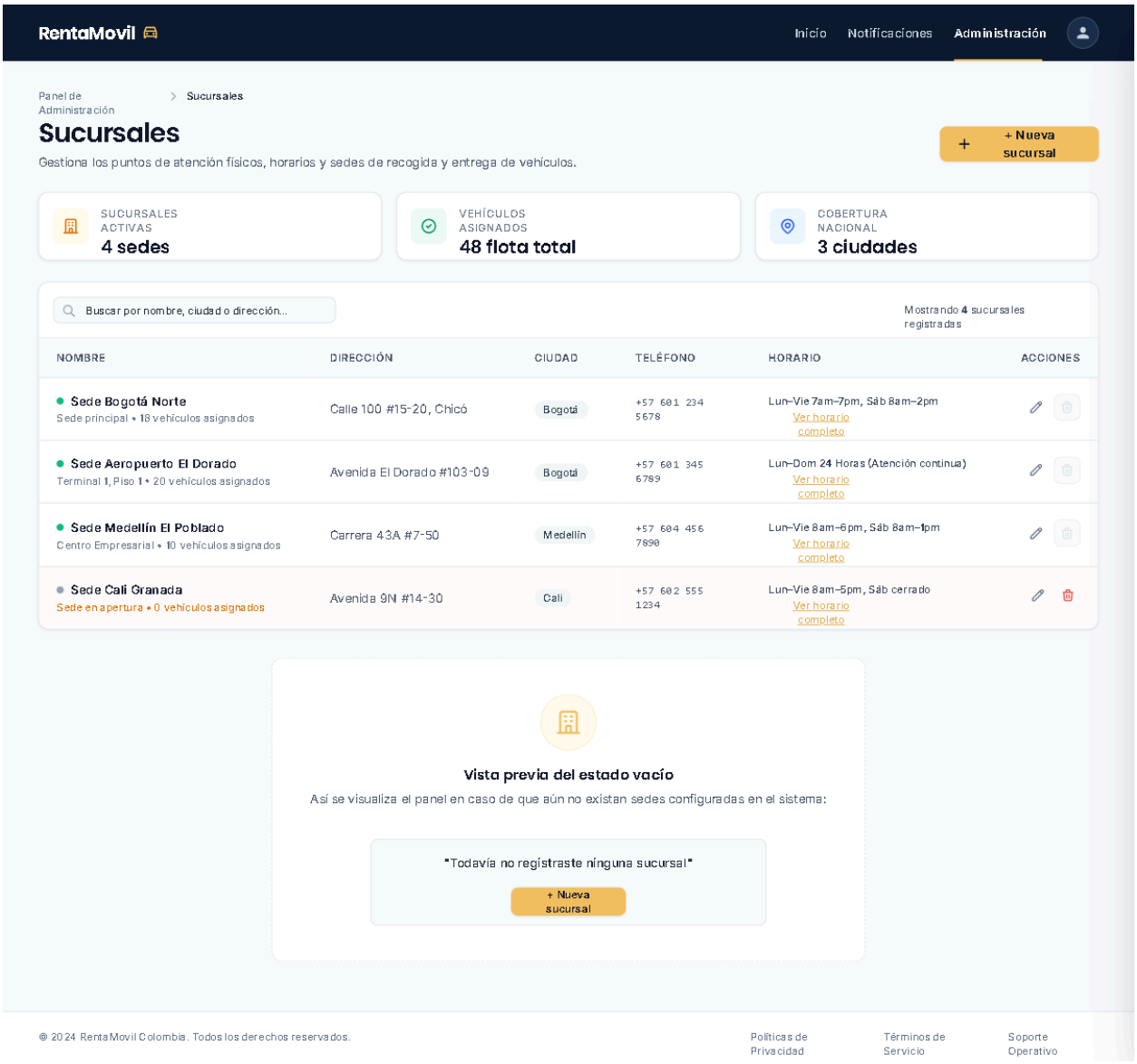
Field	Type	Nullable	Description	Constraints
audit_id	int	No		PK, IDENTITY
user_id	int	Yes	User who performed the action	FK → `user.user_id`
action	varchar(50)	No	For example, `CREATE`, `UPDATE`, `DELETE`, `LOGIN`	
affected_table	varchar(50)	No	Name of the affected table	
affected_record_id	int	Yes	Primary key of the affected record in `affected_table`	
date	datetime	No		
detail	nvarchar(max)	Yes	Flexible JSON; its structure depends on `action` and `affected_table`, not on a fixed set of columns	Validate with `ISJSON()`

Appendix B. Graphical user interfaces

The 23 available interface images and the 21 documented routes are included, with some views shared by role.

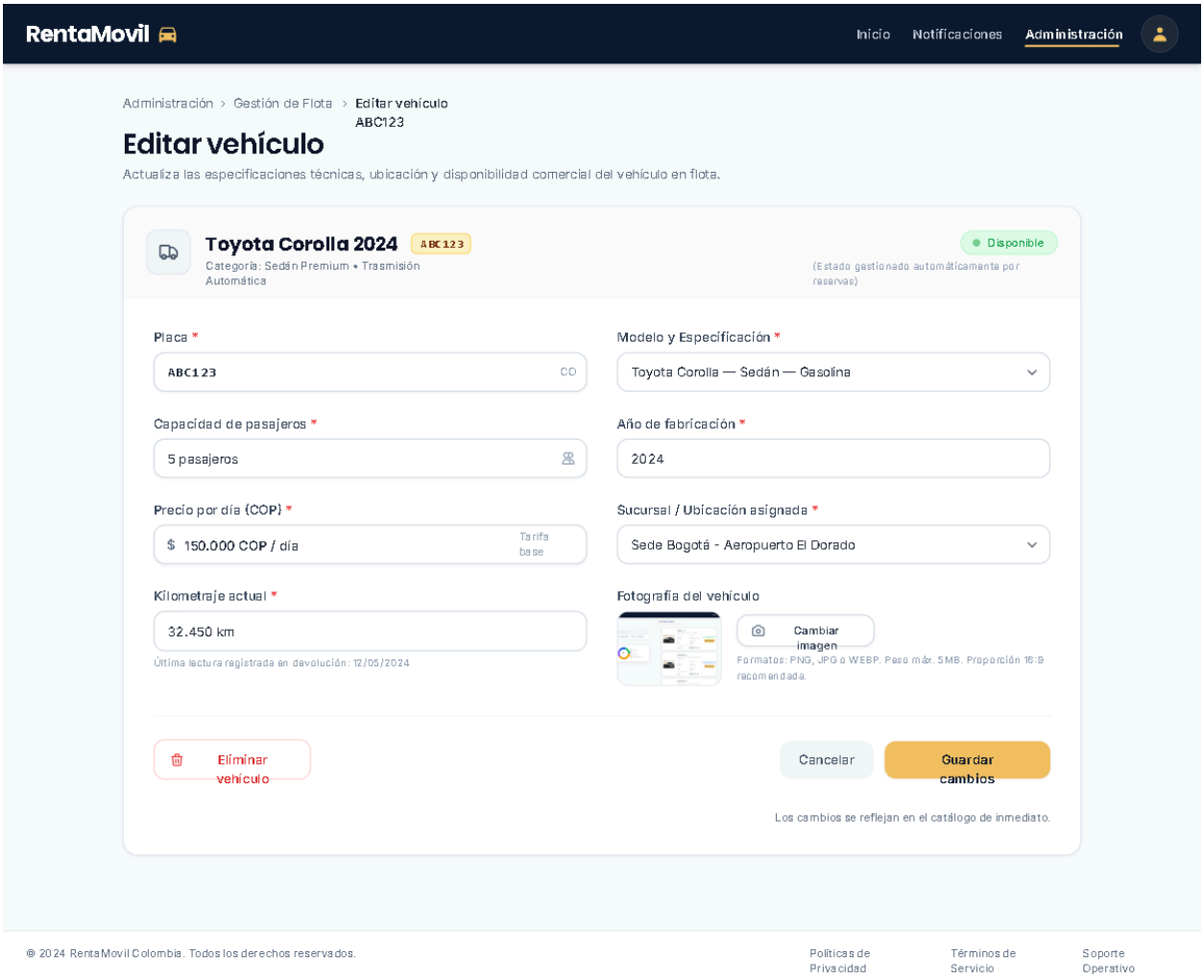
The complementary inventory identifies design PDF files available in the source package.

Figure 1. *Branch administration*



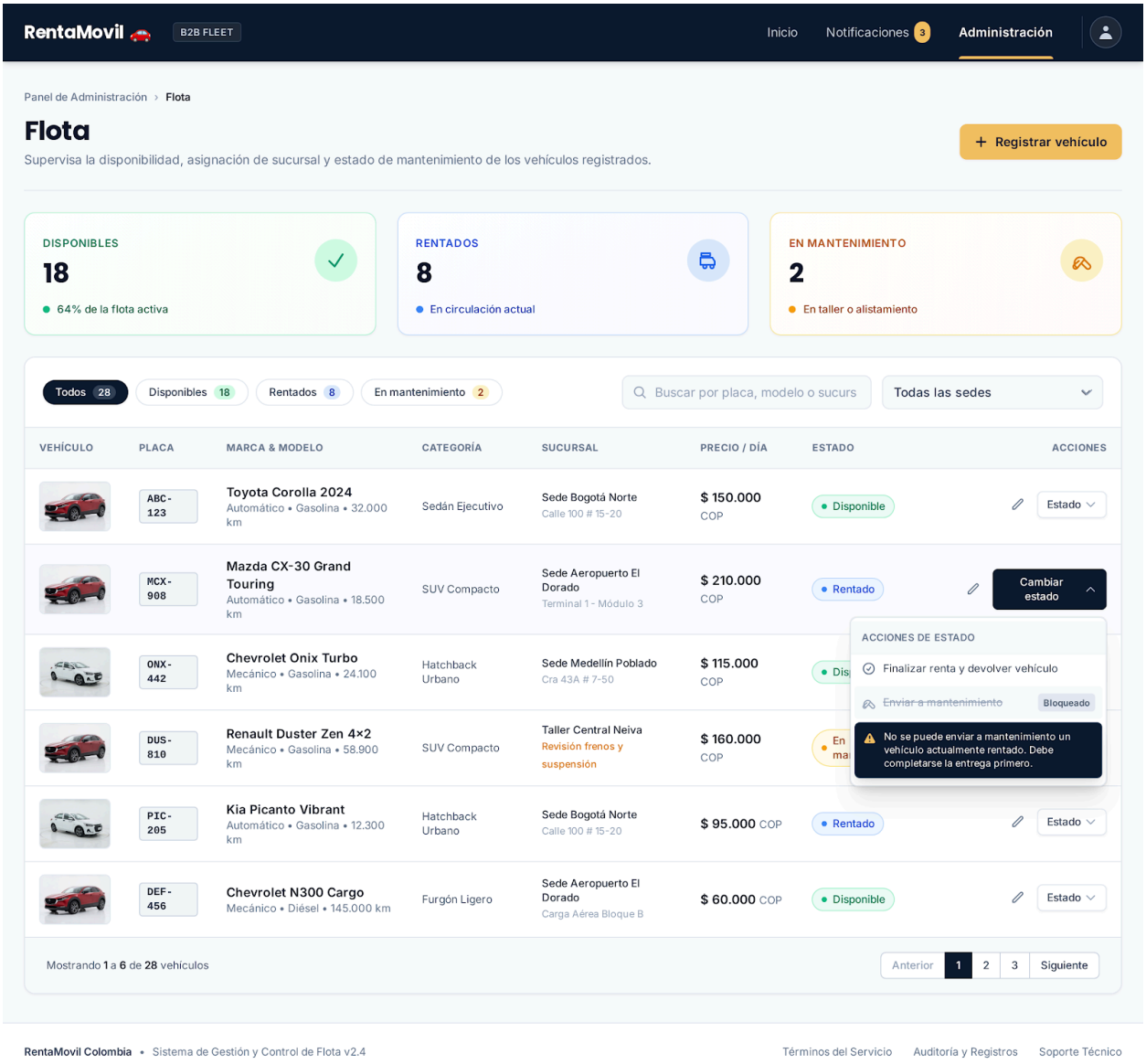
Source: RentaMovil visual prototype, 2026.

Figure 2. *Vehicle editing*



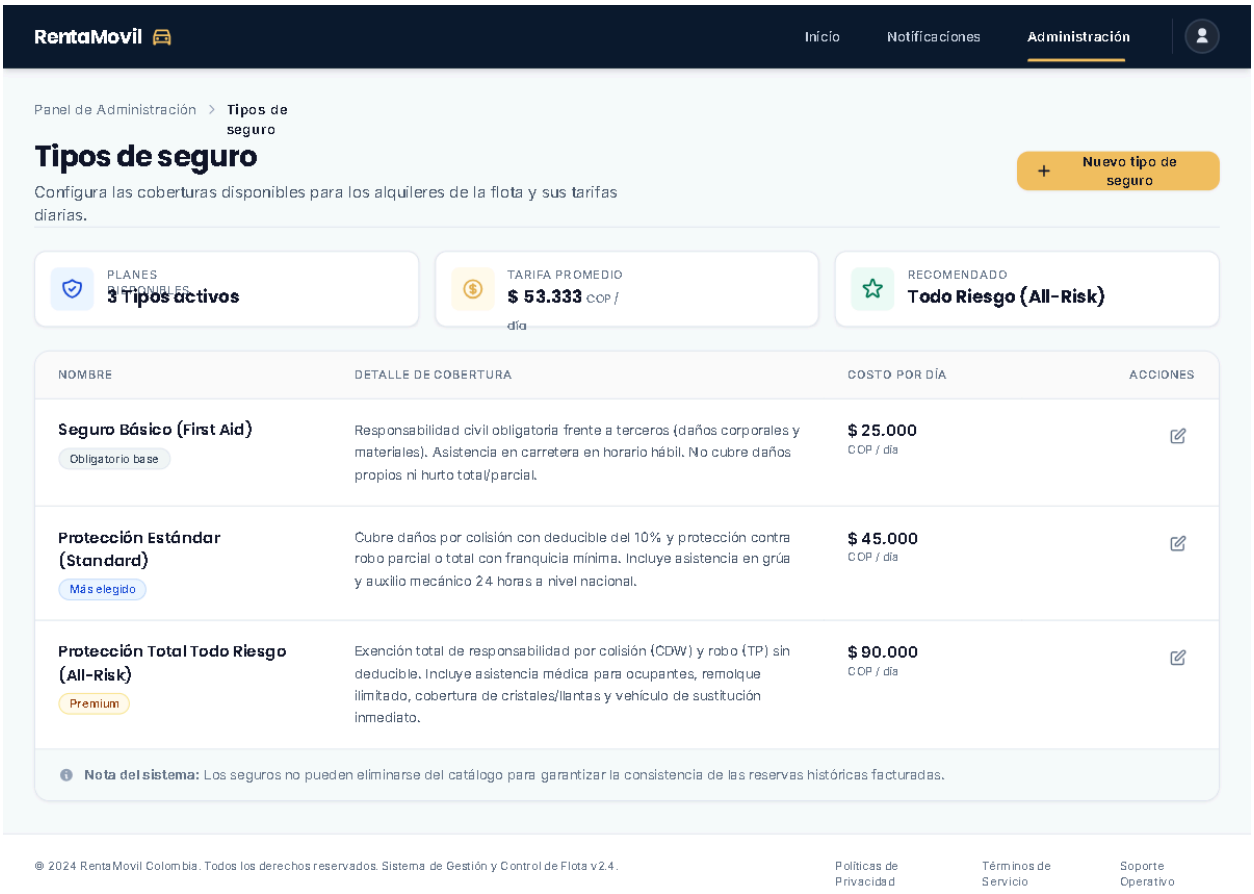
Source: RentaMovil visual prototype, 2026.

Figure 3. *Fleet dashboard*



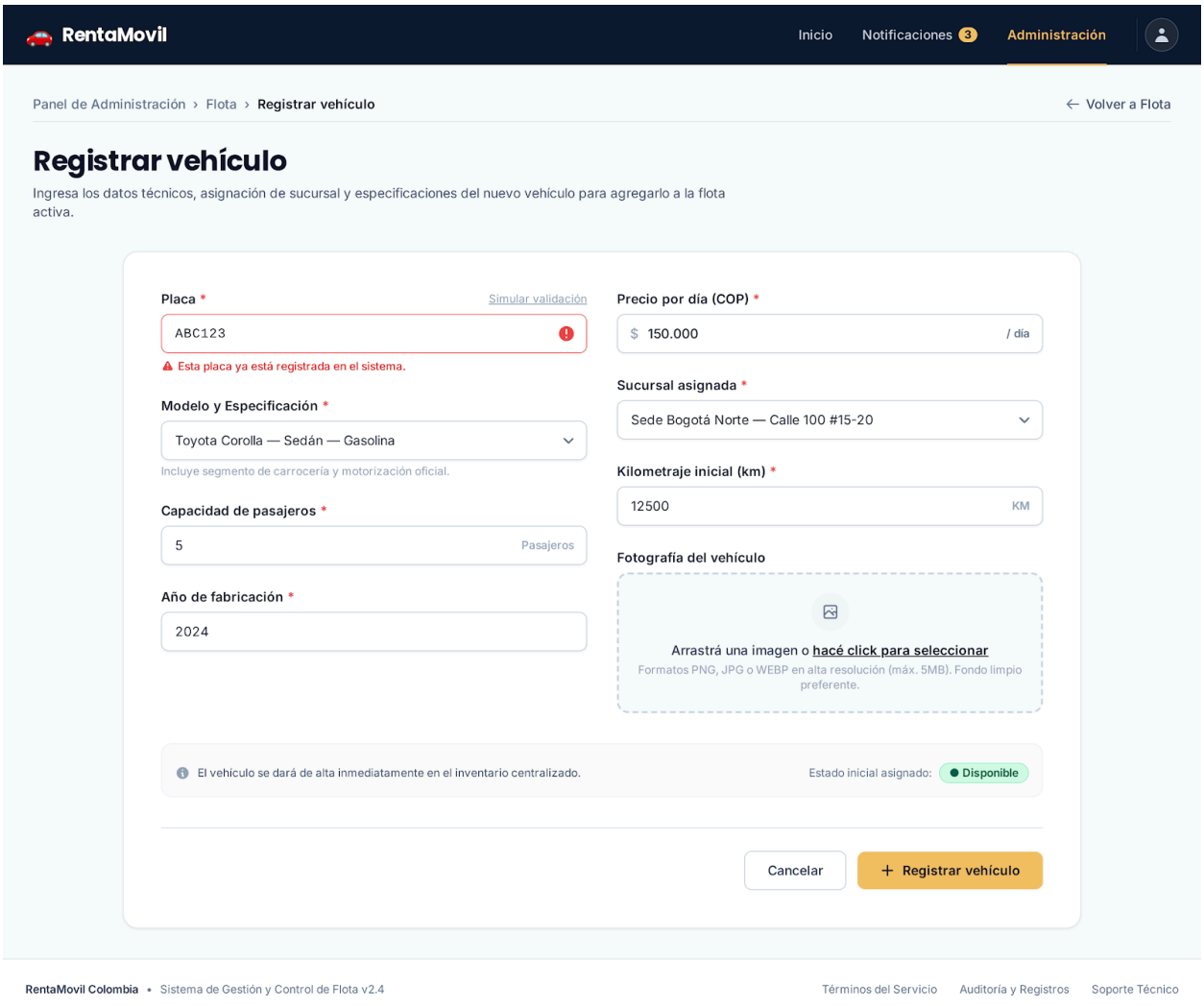
Source: RentaMovil visual prototype, 2026.

Figure 4. *Insurance types*



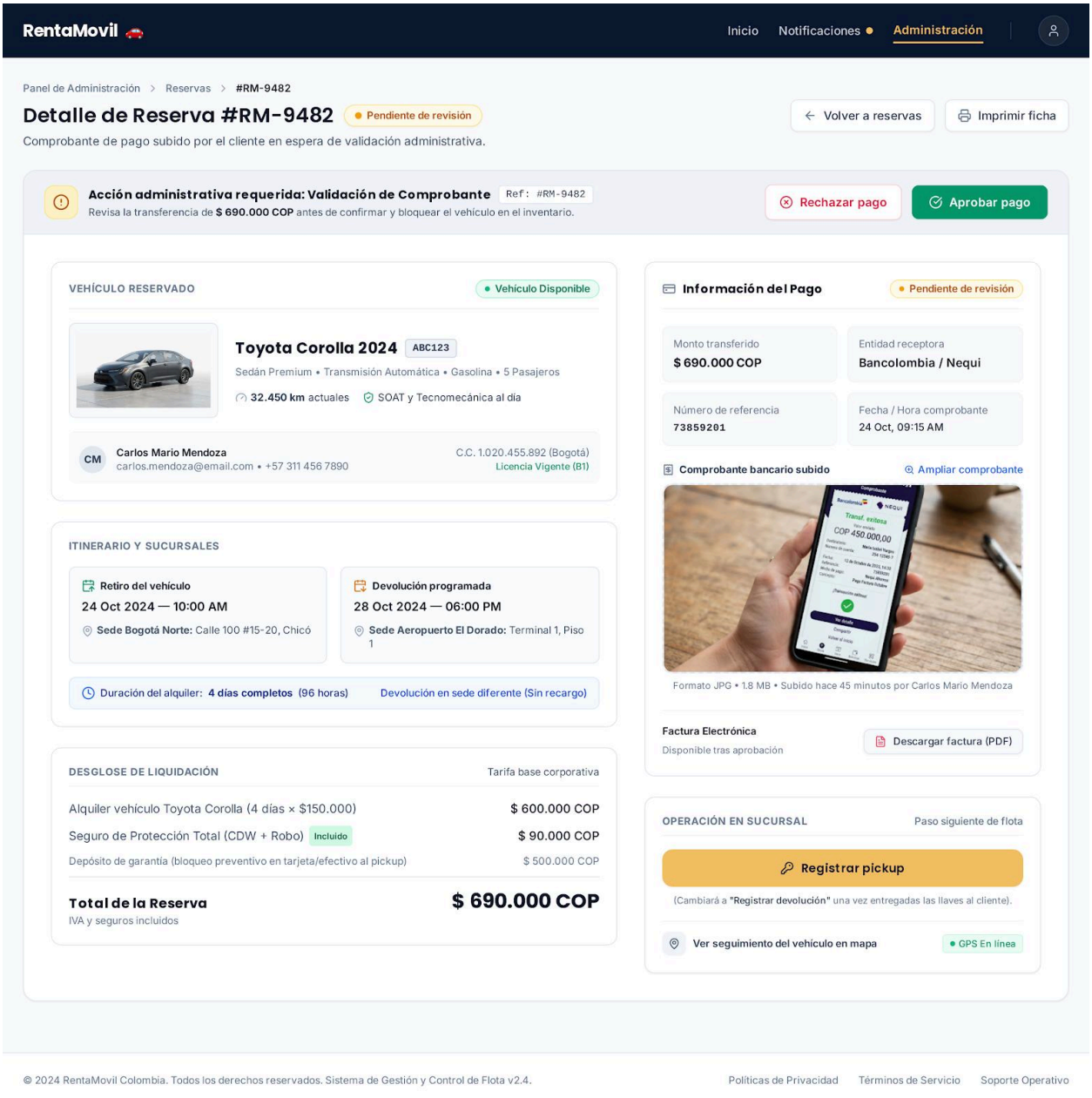
Source: RentaMovil visual prototype, 2026.

Figure 5. *Vehicle registration*



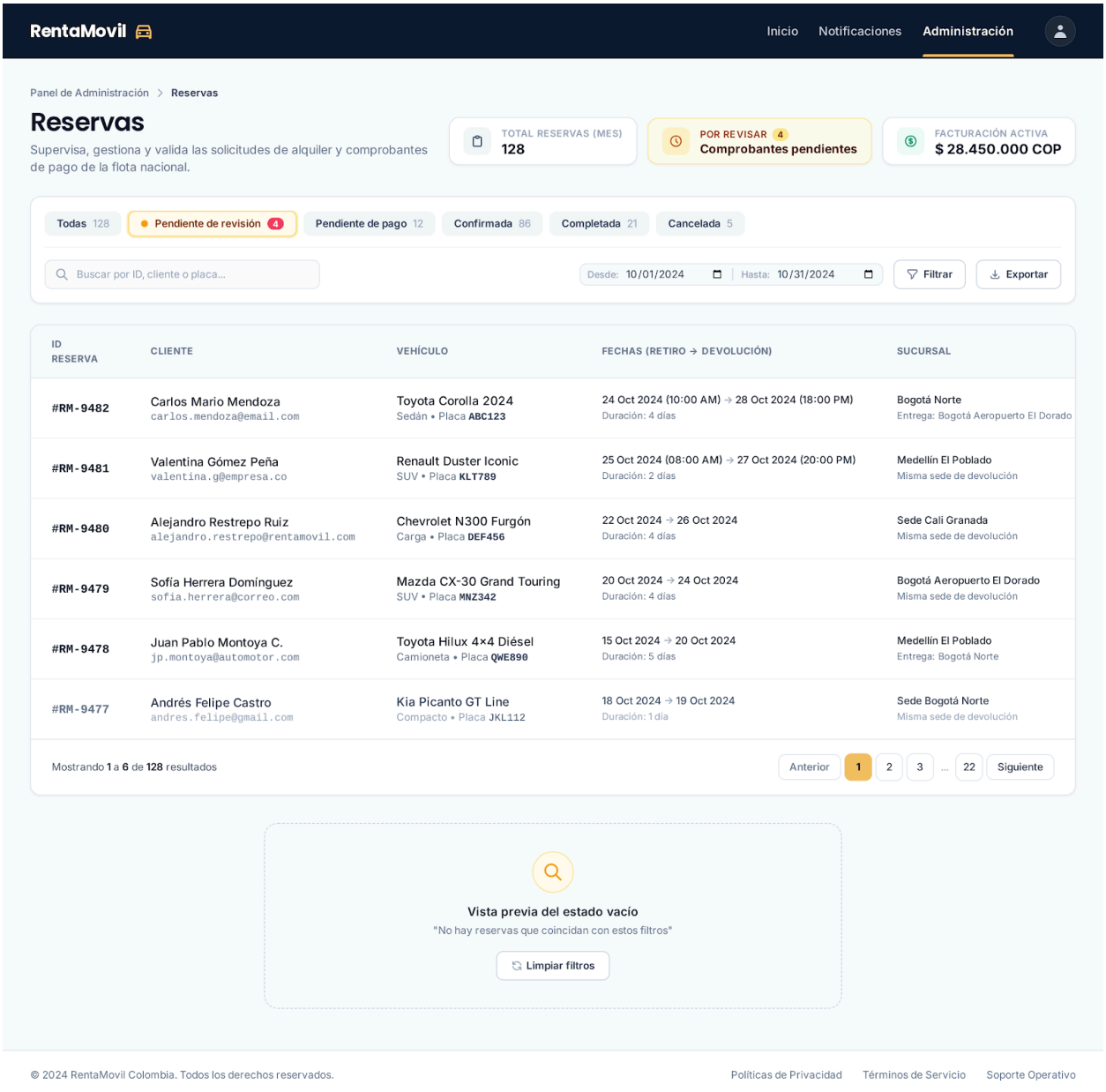
Source: RentaMovil visual prototype, 2026.

Figure 6. *Booking detail*



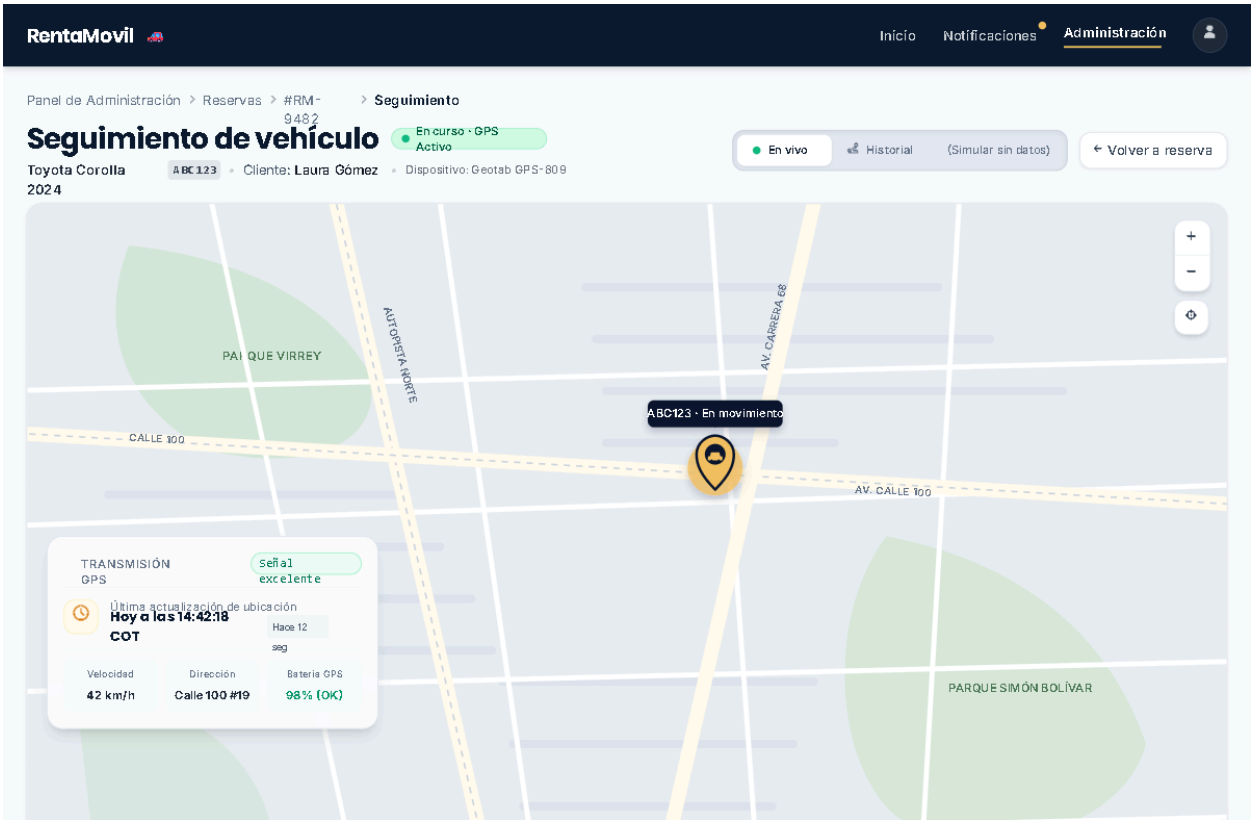
Source: RentaMovil visual prototype, 2026.

Figure 7. *Bookings dashboard*



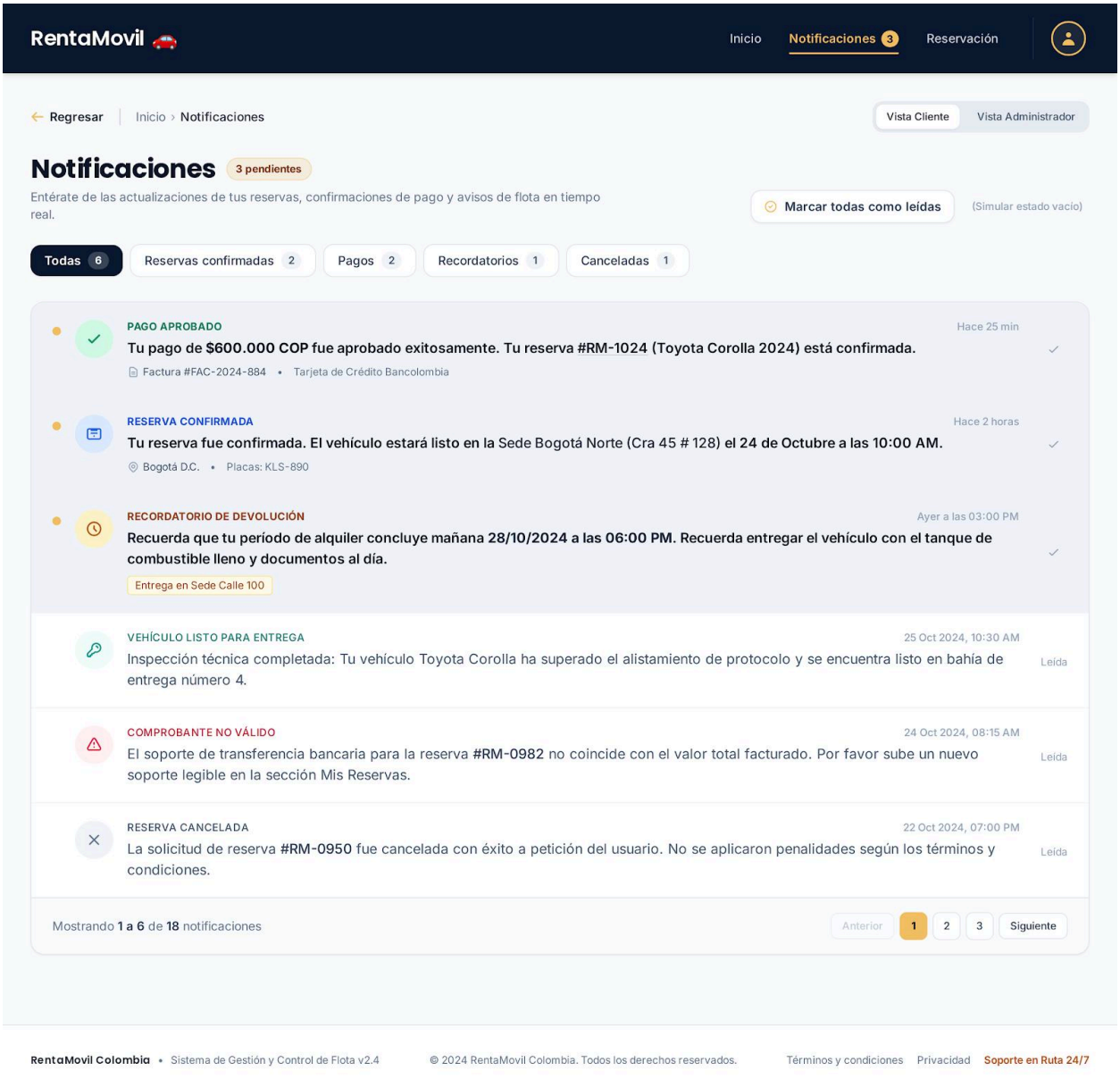
Source: RentaMovil visual prototype, 2026.

Figure 8. *Vehicle tracking*



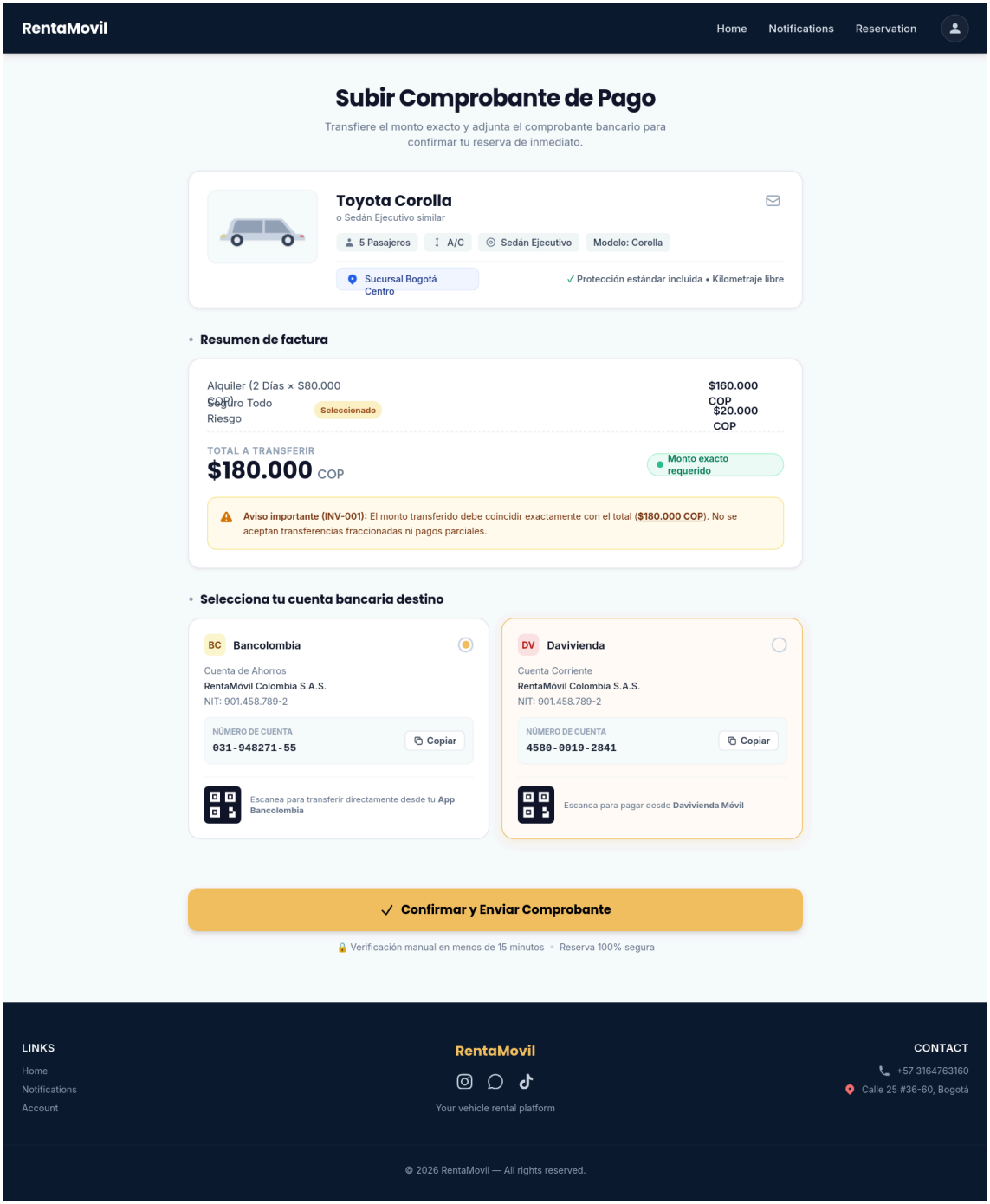
Source: RentaMovil visual prototype, 2026.

Figure 9. *Client notifications*



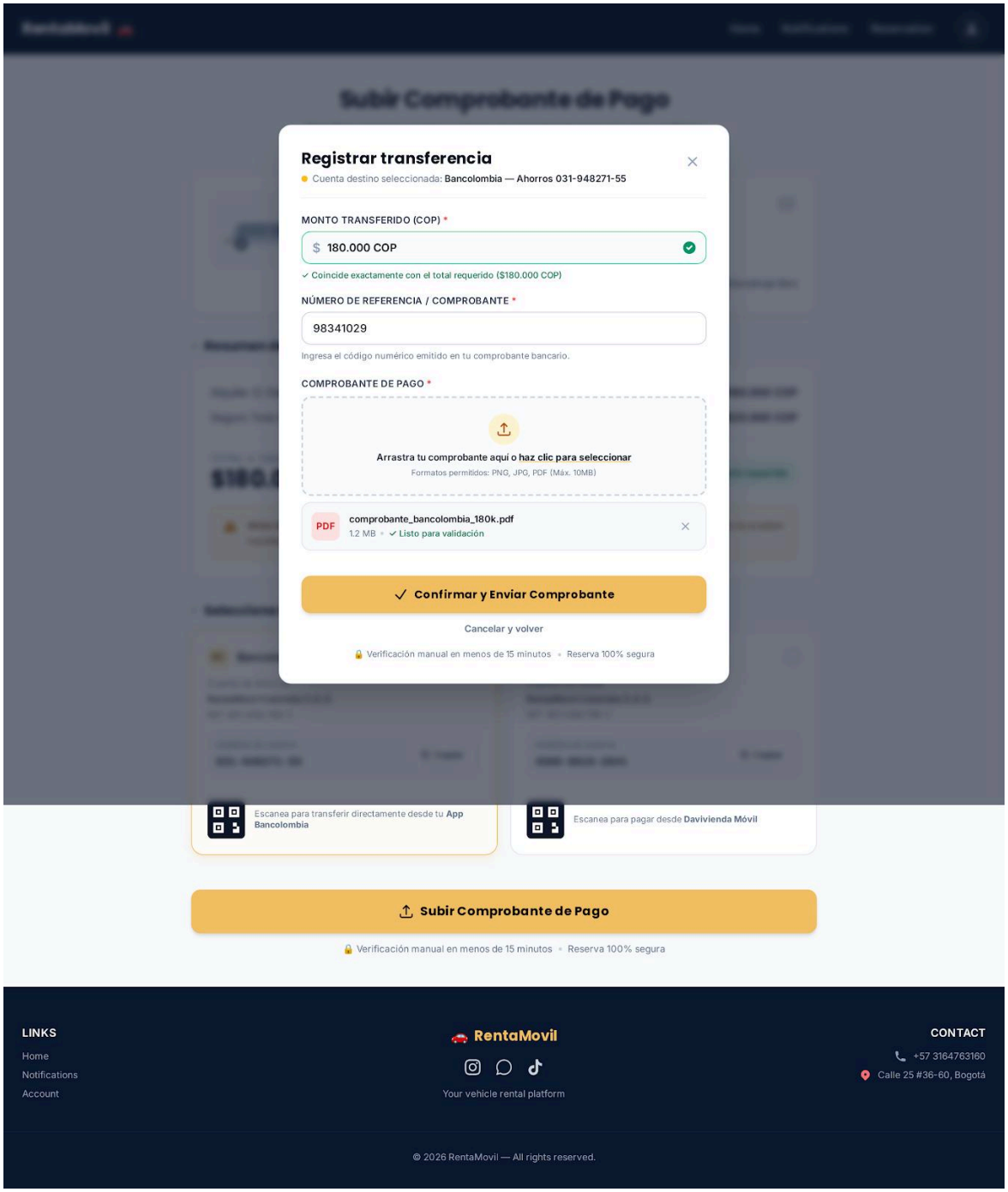
Source: RentaMovil visual prototype, 2026.

Figure 10. *Payment upload*



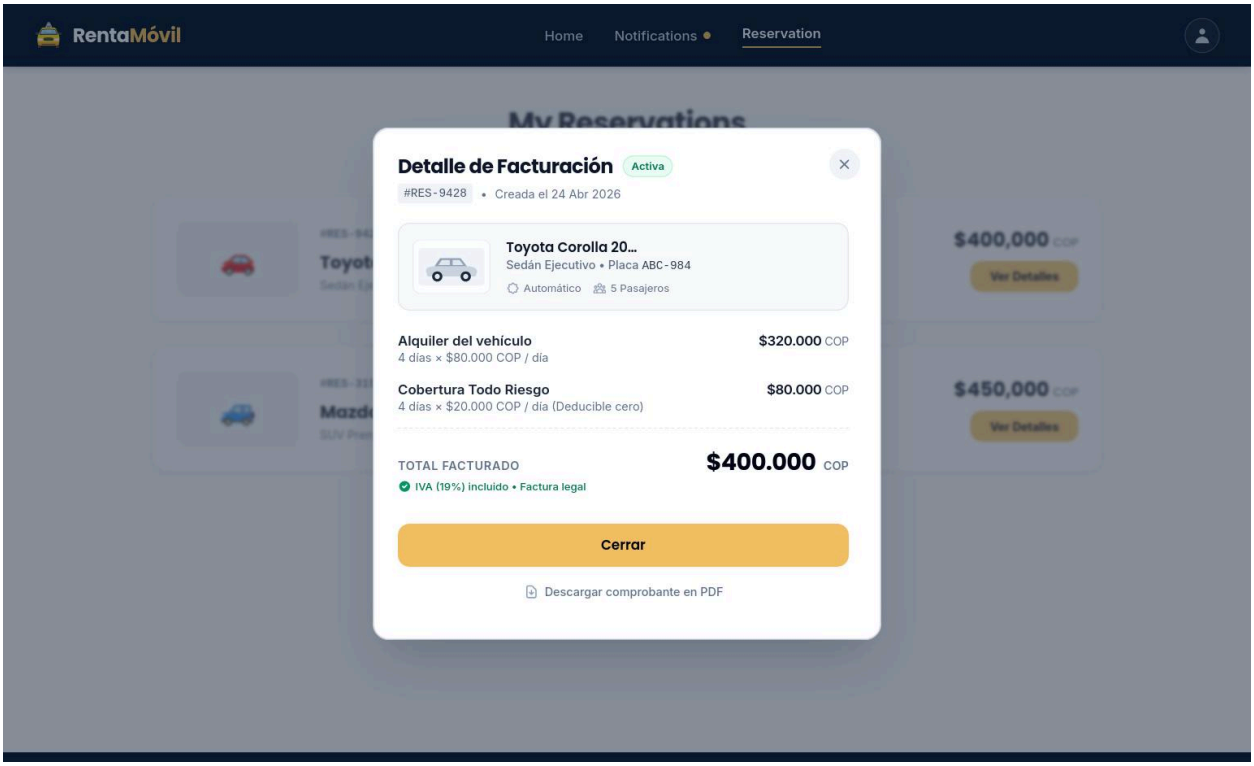
Source: RentaMovil visual prototype, 2026.

Figure 11. *Transfer receipt*



Source: RentaMovil visual prototype, 2026.

Figure 12. *Booking detail and invoice*



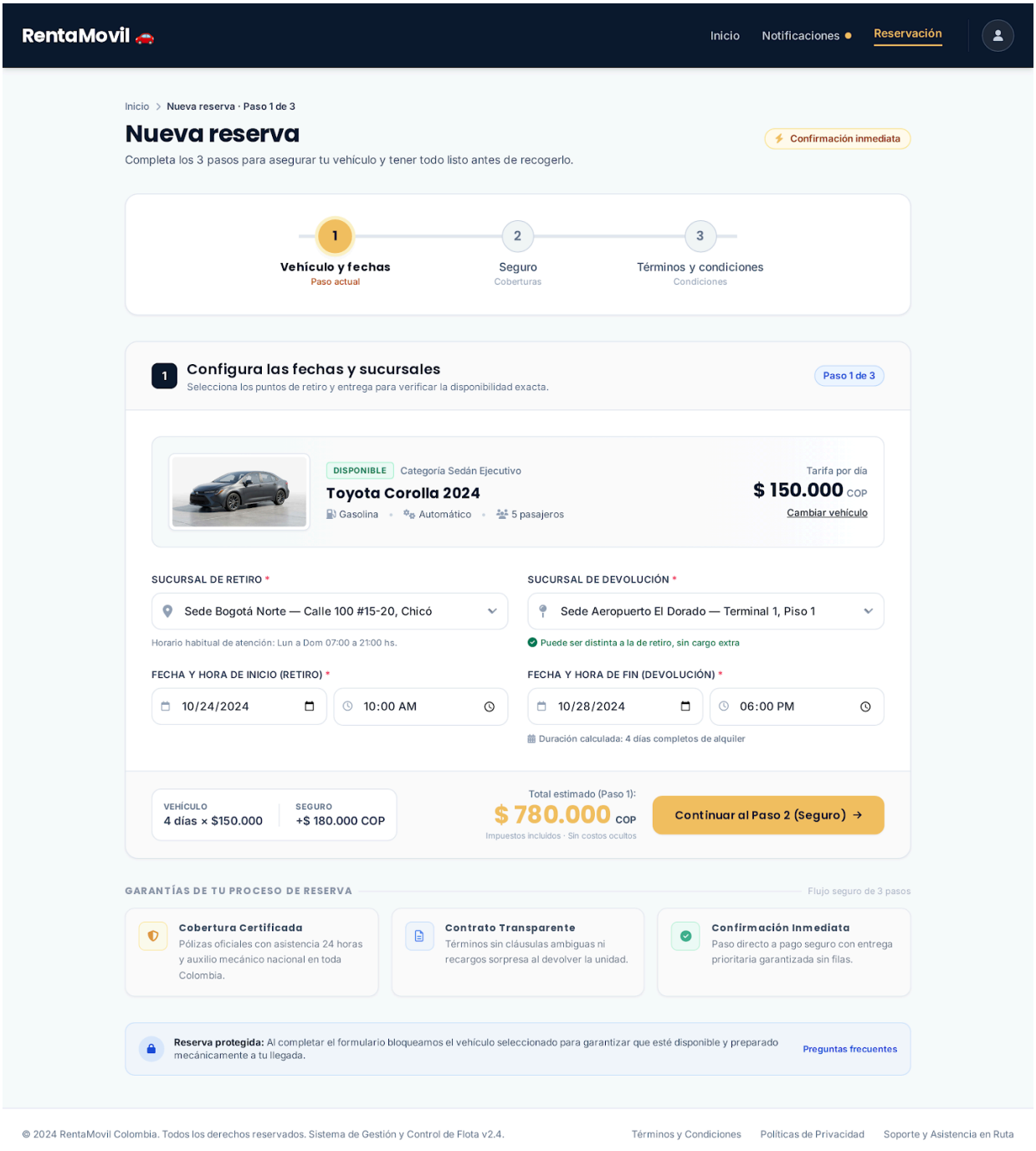
Source: RentaMovil visual prototype, 2026.

Figure 13. *My bookings*



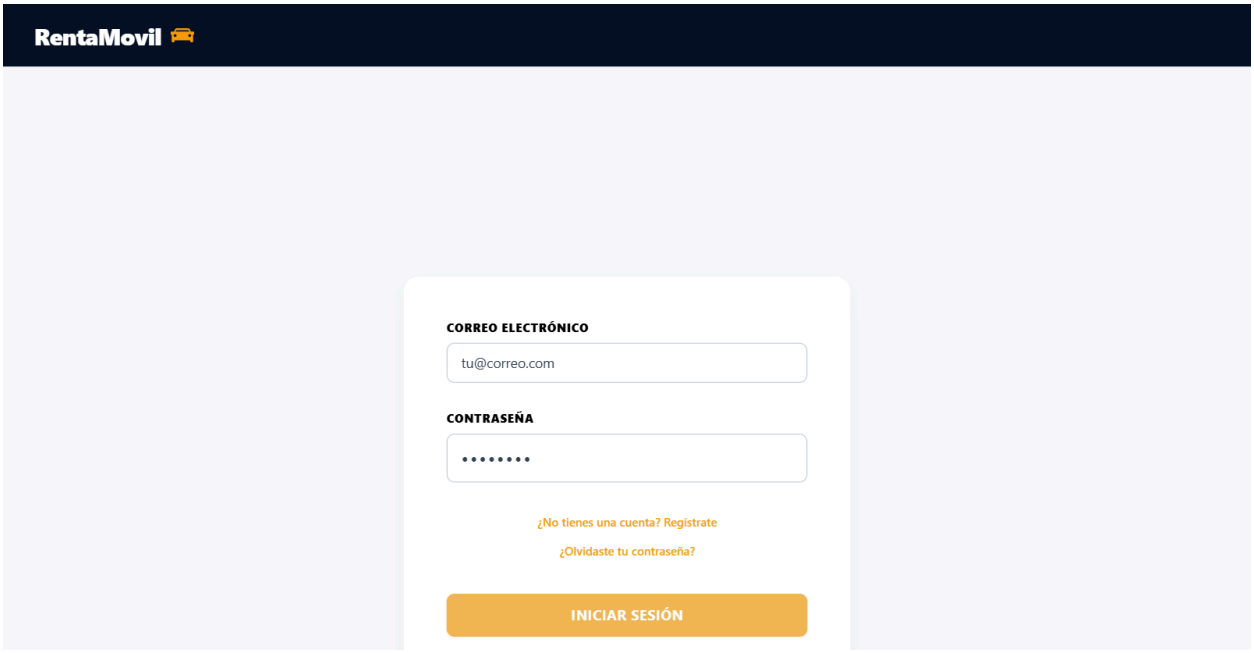
Source: RentaMovil visual prototype, 2026.

Figure 14. *Booking wizard*



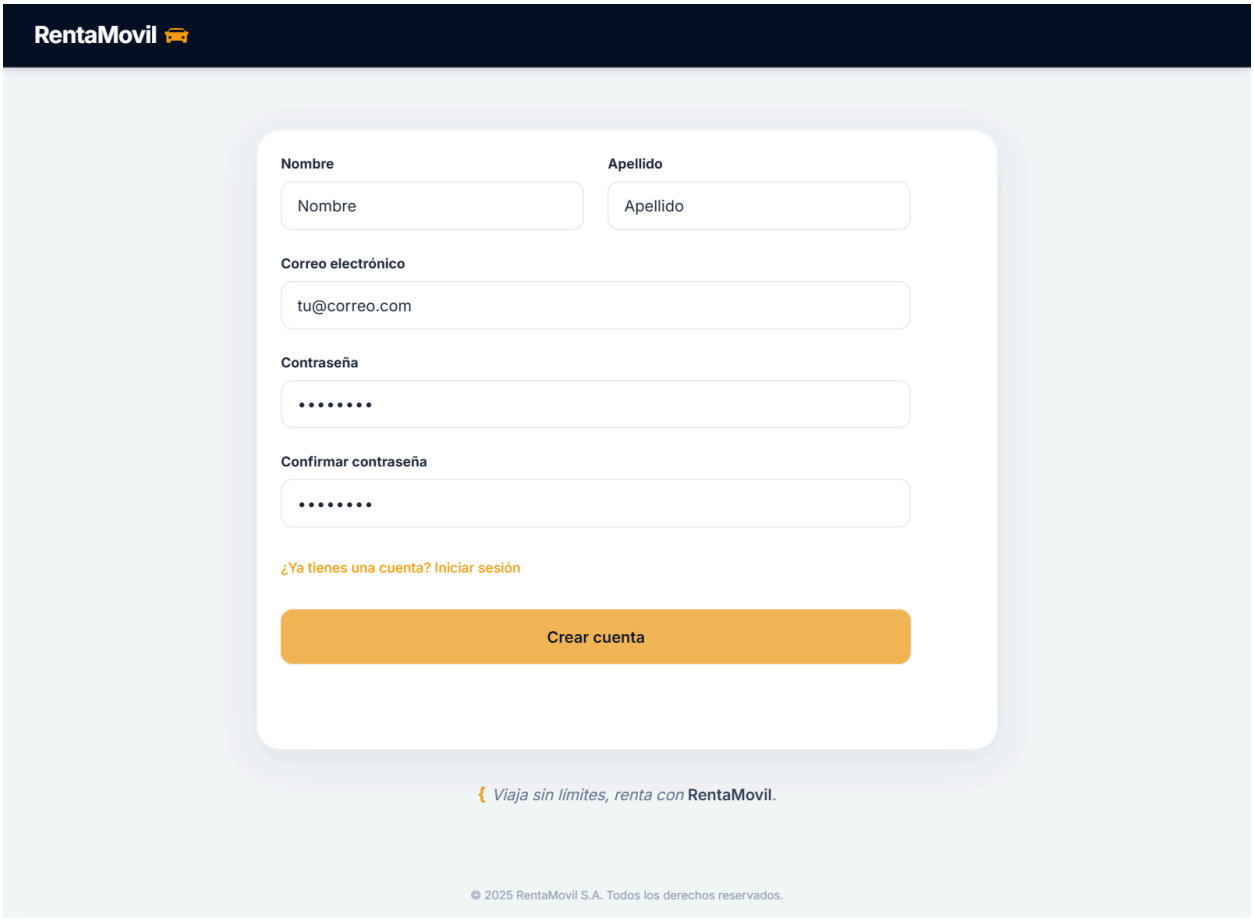
Source: RentaMovil visual prototype, 2026.

Figure 15. *Sign-in*



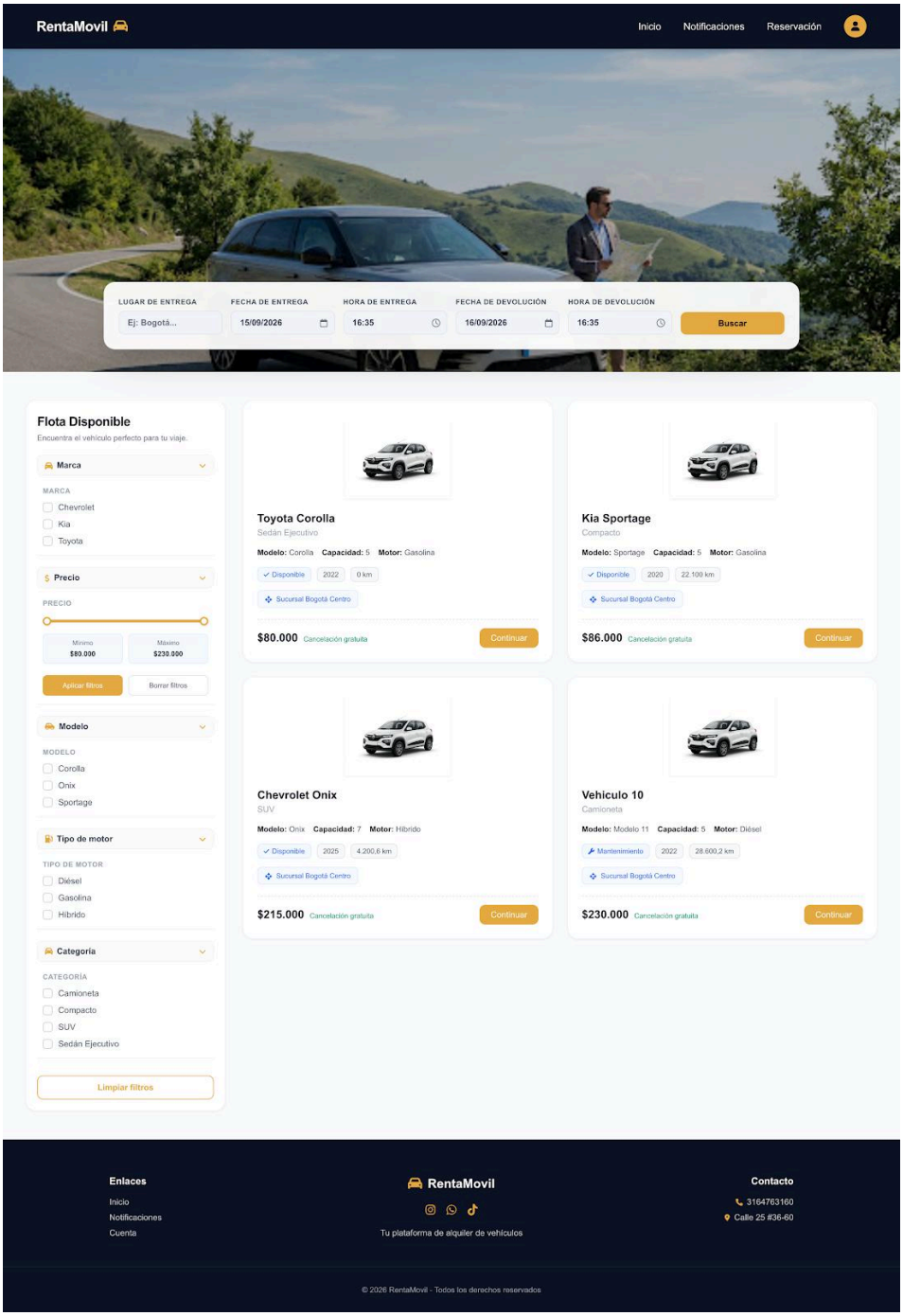
Source: RentaMovil visual prototype, 2026.

Figure 16. *User registration*



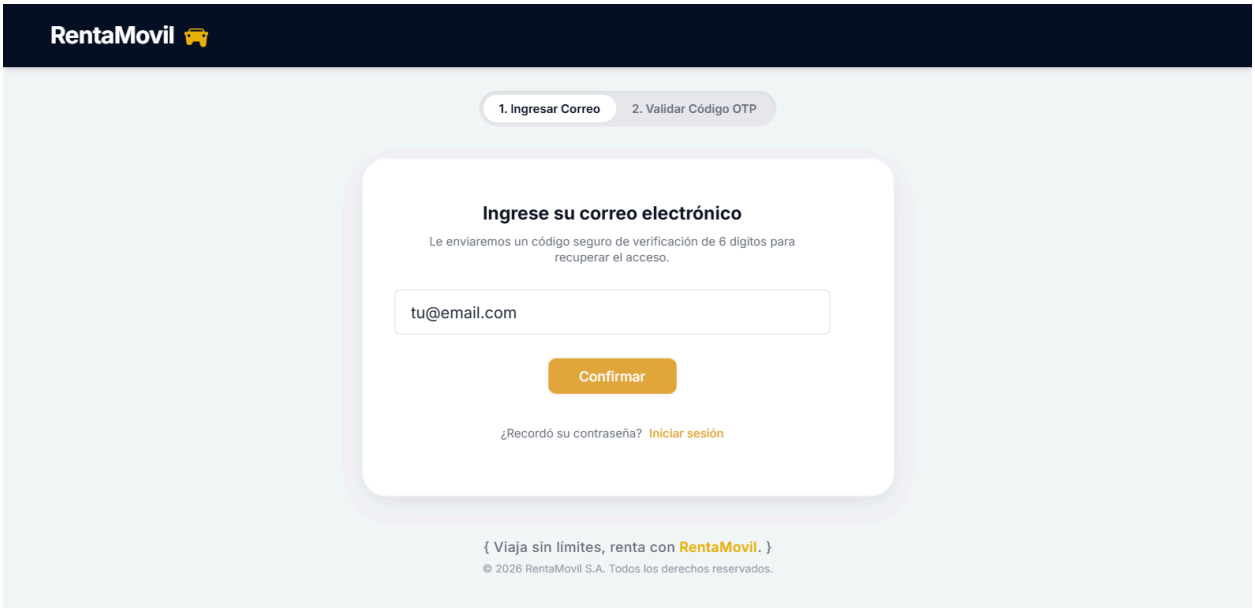
Source: RentaMovil visual prototype, 2026.

Figure 17. *Vehicle catalog*



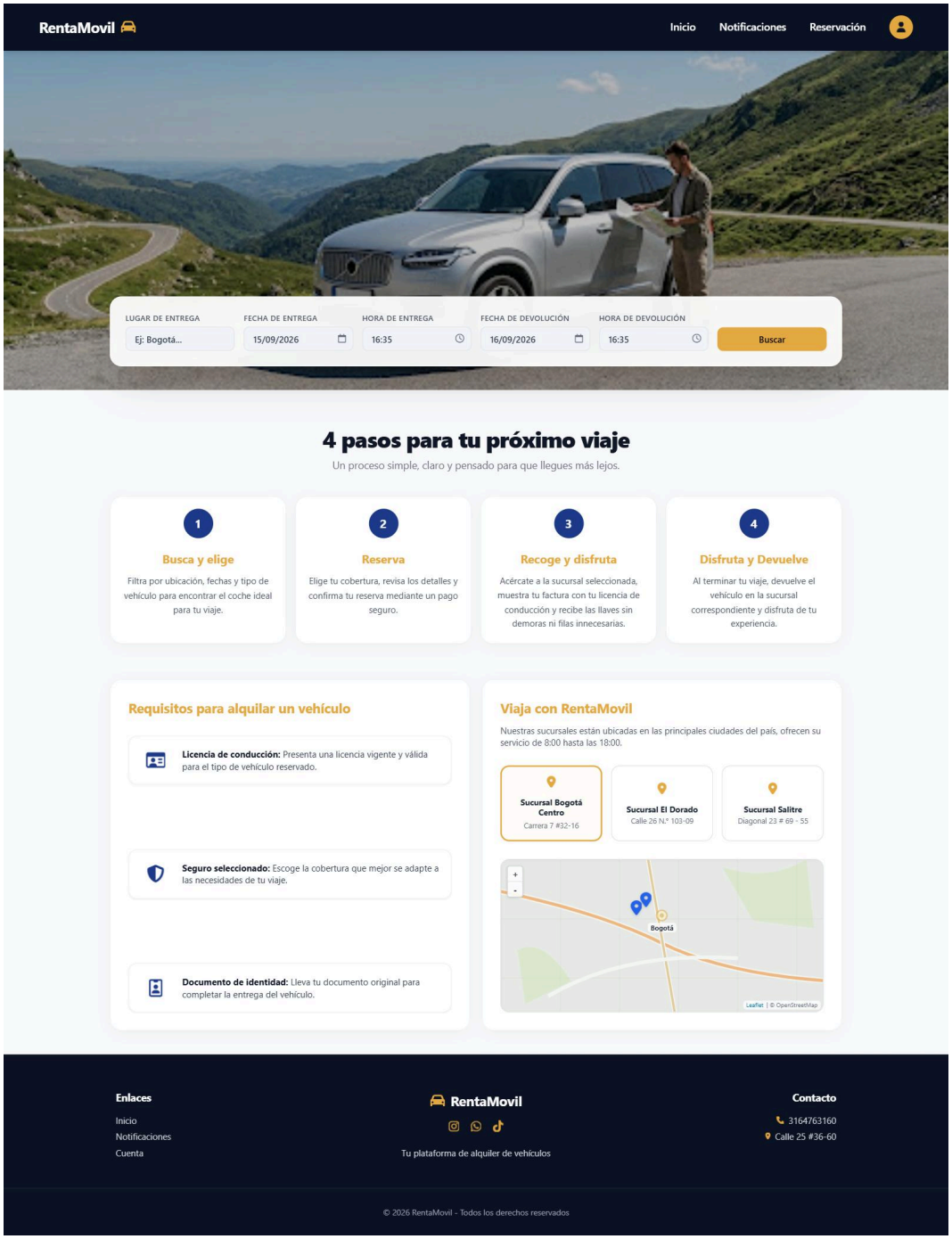
Source: RentaMovil visual prototype, 2026.

Figure 18. *Password recovery*



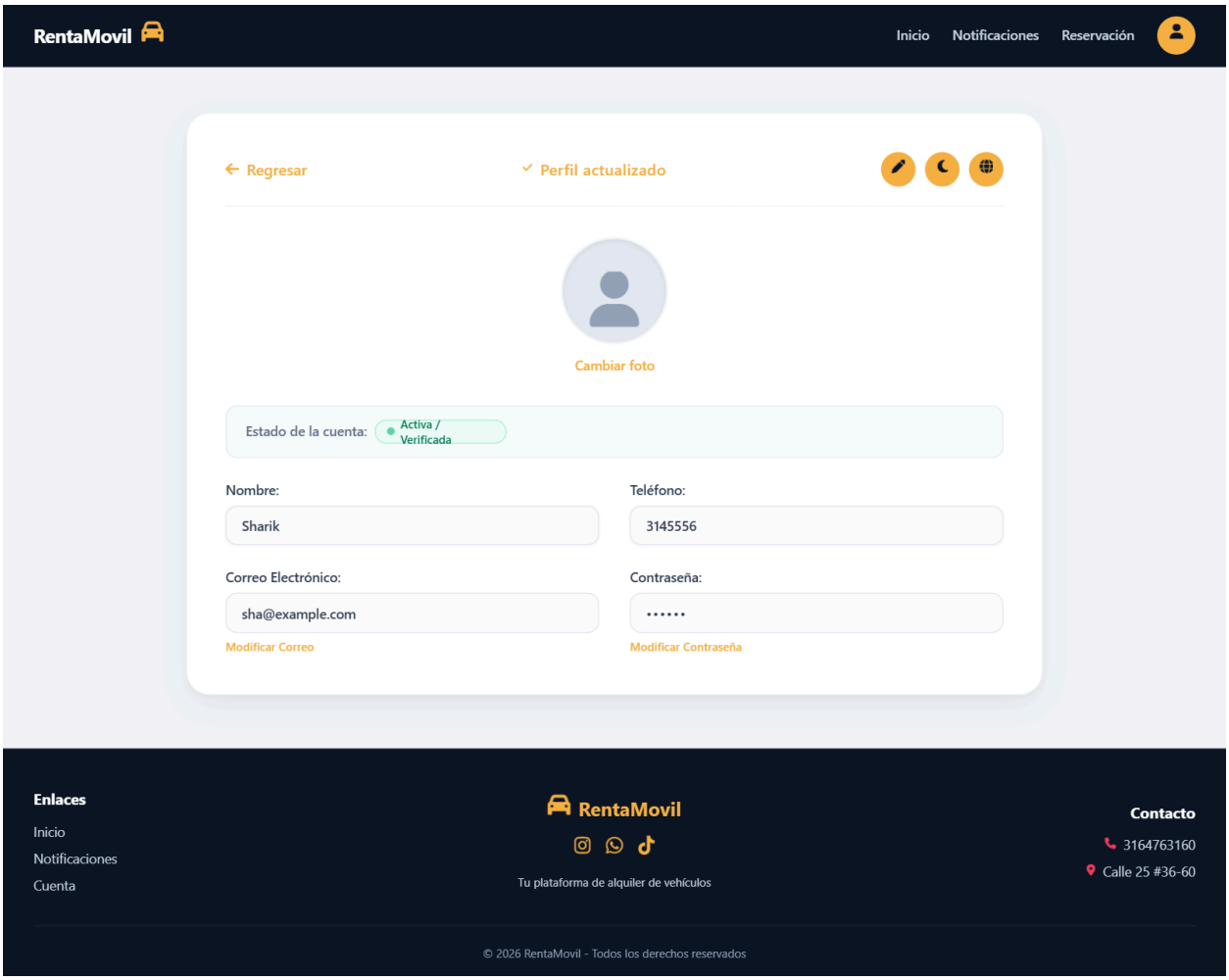
Source: RentaMovil visual prototype, 2026.

Figure 19. *Home*



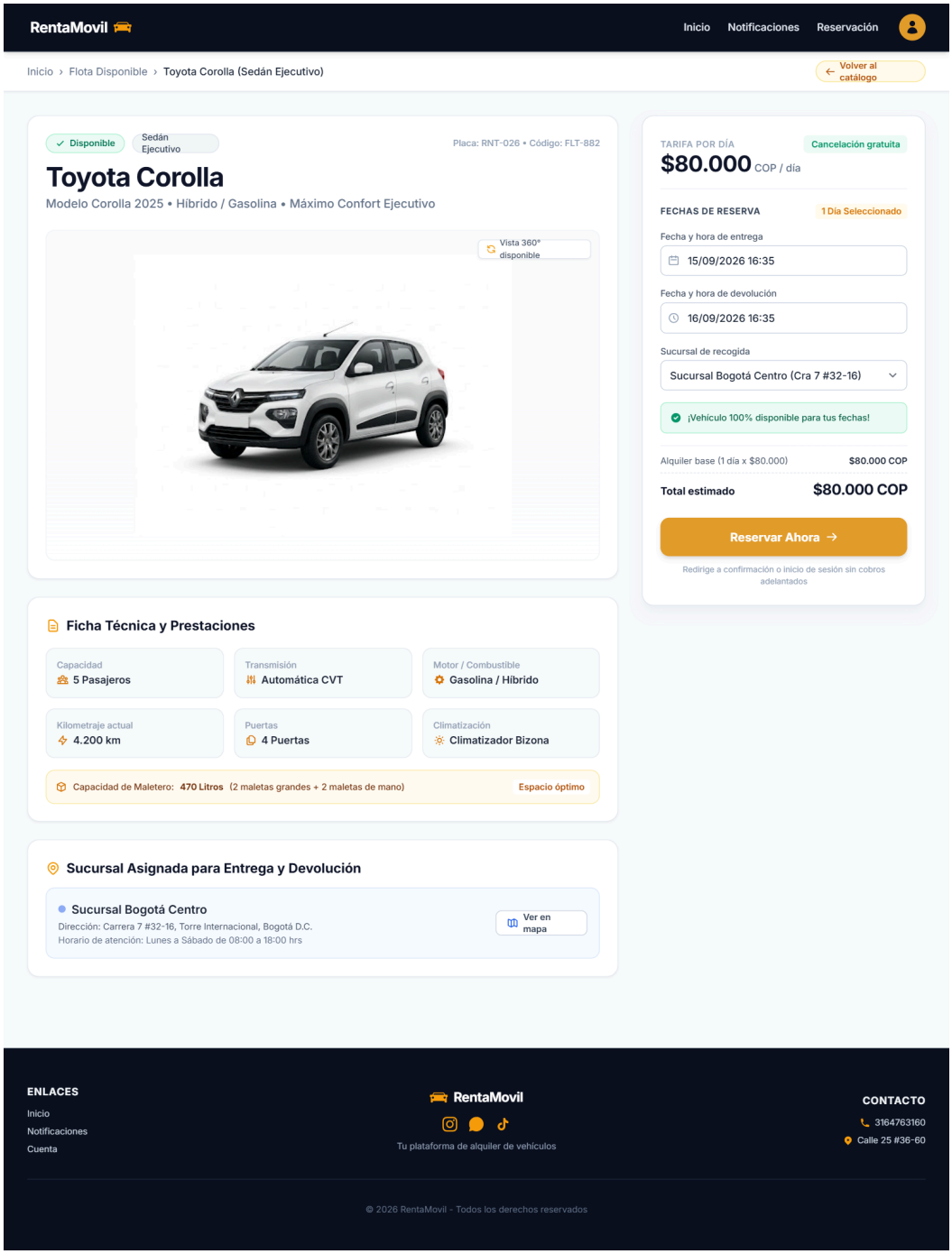
Source: RentaMovil visual prototype, 2026.

Figure 20. *Profile*



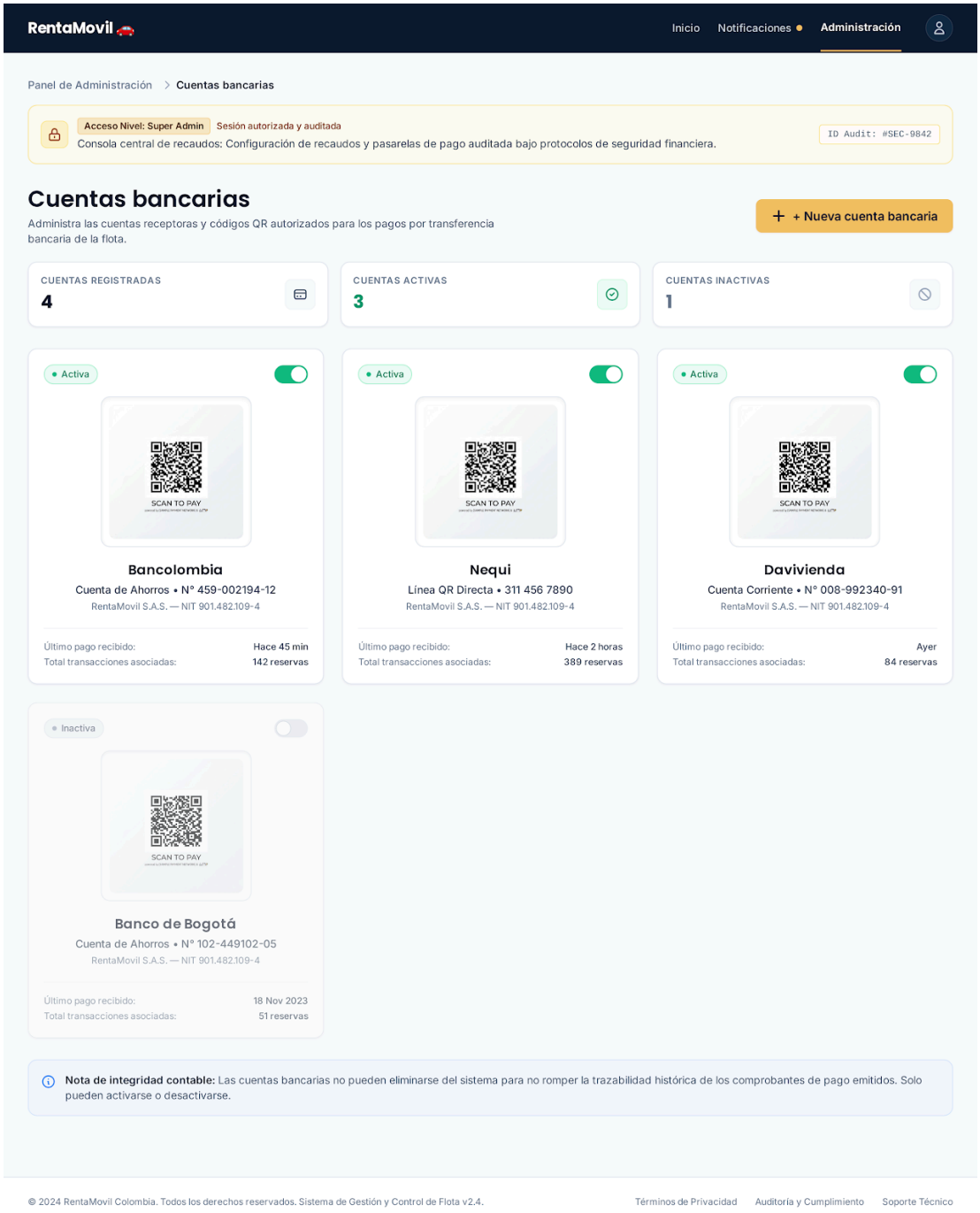
Source: RentaMovil visual prototype, 2026.

Figure 21. *Vehicle detail*



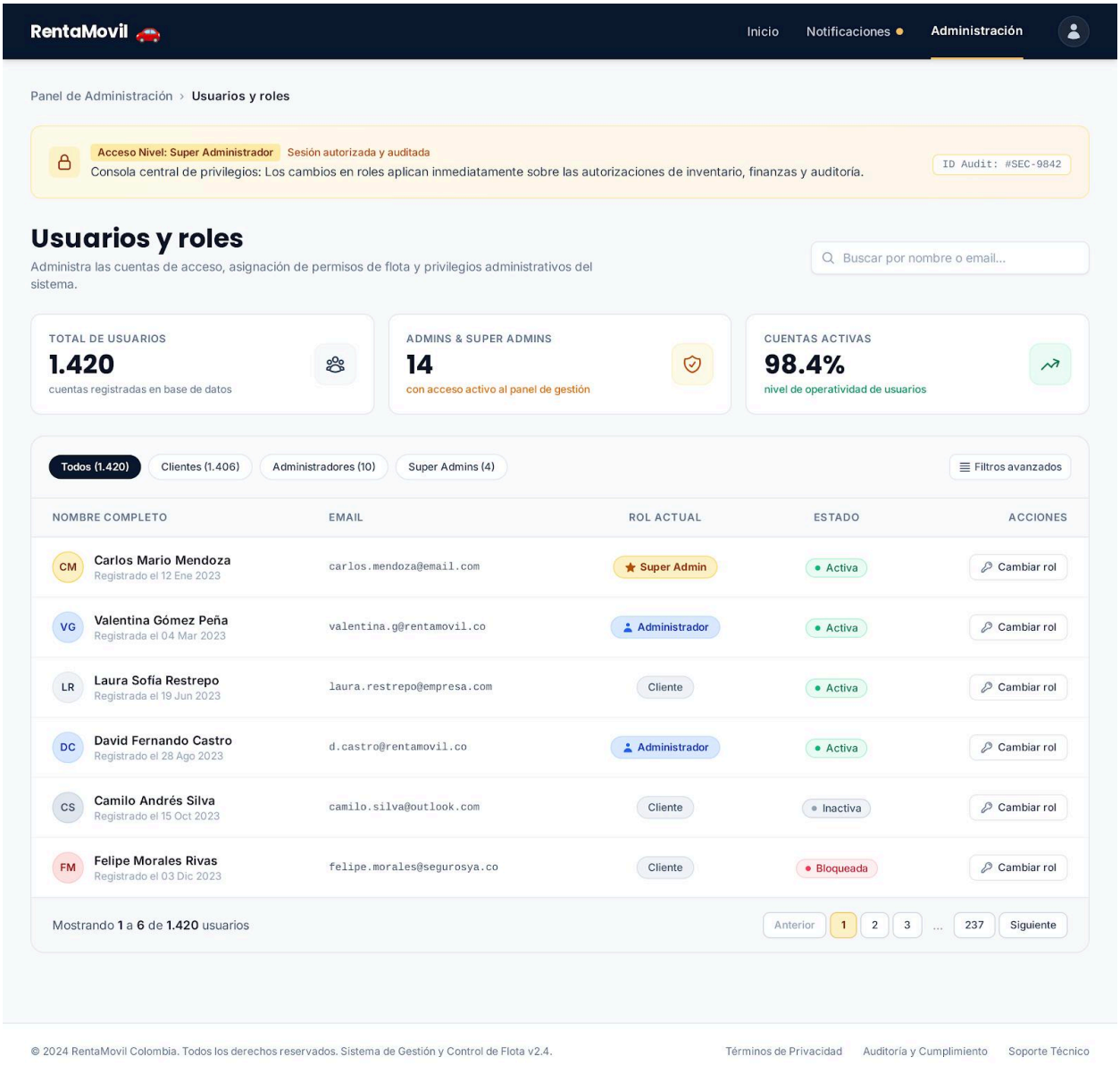
Source: RentaMovil visual prototype, 2026.

Figure 22. *Bank accounts*



Source: RentaMovil visual prototype, 2026.

Figure 23. *User and role administration*



Source: RentaMovil visual prototype, 2026.

Appendix C. Document inventory

PDF design files contained in the RentaMovil.zip source package when this report was prepared. The original package is kept outside this repository.

Table 34.
Design PDF files in the source package

File	Type
Panel Admin.pdf	Design or screen PDF
Footer.pdf	Design or screen PDF
Nav Bar.pdf	Design or screen PDF
checkStatus.pdf	Design or screen PDF
addVehicle.pdf	Design or screen PDF
inventary.pdf	Design or screen PDF
registerMaintenance.pdf	Design or screen PDF
historyMaintenance.pdf	Design or screen PDF
location.pdf	Design or screen PDF
checkUSer.pdf	Design or screen PDF
notifications.pdf	Design or screen PDF
contractHistory.pdf	Design or screen PDF
count.pdf	Design or screen PDF
page.pdf	Design or screen PDF
Overlay.pdf	Design or screen PDF
reservation.pdf	Design or screen PDF
historyLocation.pdf	Design or screen PDF
NewContract.pdf	Design or screen PDF
P.A V.pdf	Design or screen PDF
P.A U.pdf	Design or screen PDF
P.A M.pdf	Design or screen PDF
P.A C.pdf	Design or screen PDF
Home Admin V.pdf	Design or screen PDF
Overlay-1.pdf	Design or screen PDF
Overlay-2.pdf	Design or screen PDF
Overlay-3.pdf	Design or screen PDF

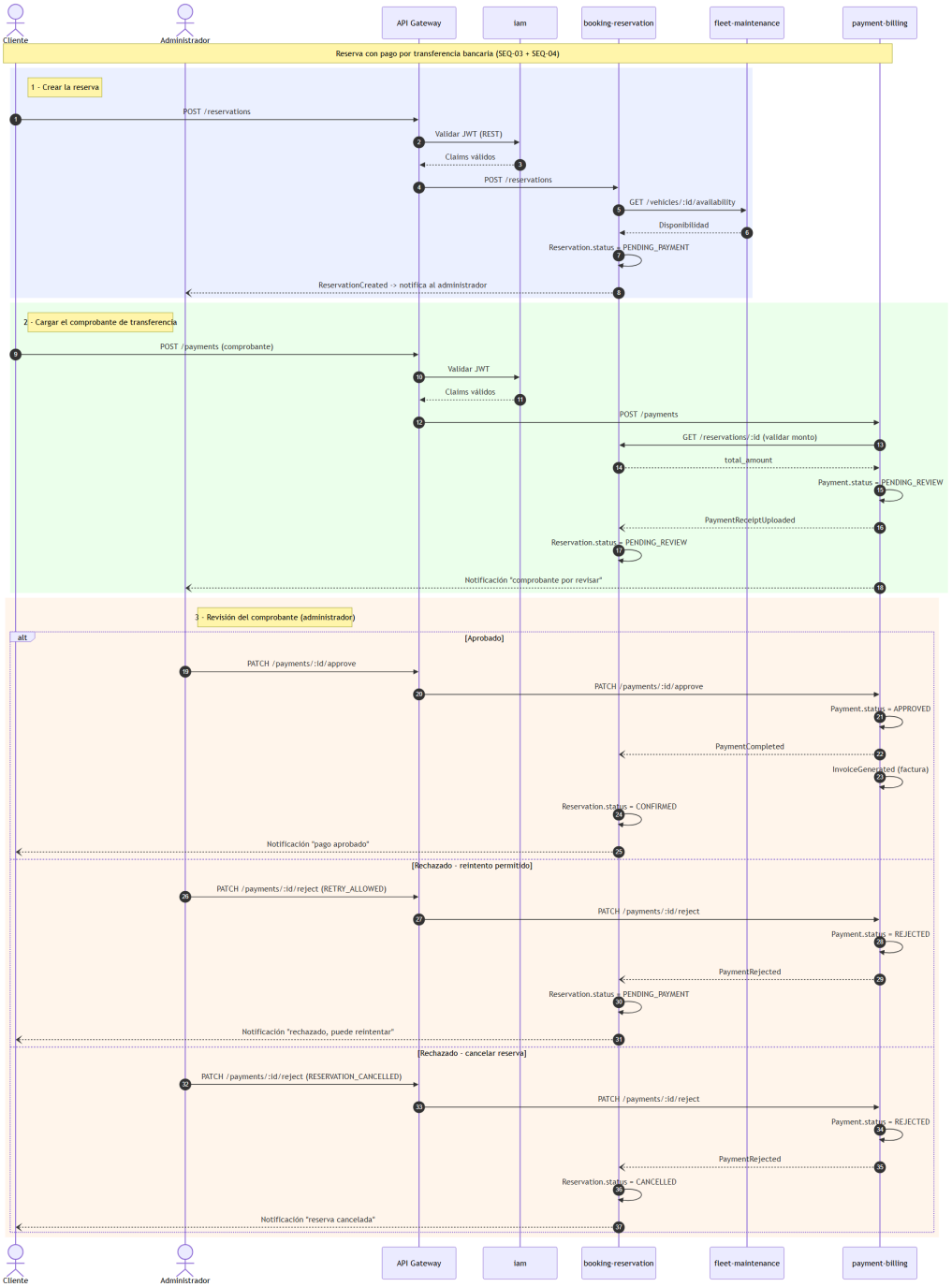
Appendix D. Support diagrams

These diagrams were prepared only with information already documented in the project repositories; no data lacking a source was added. Sources: the analysis document `02-documentacion-analisis-software` (sequence SEQ and state EST diagrams), `rtm-docs/02-domain/domain-events.md`, `rtm-docs/02-domain/entities-and-rules.md`, `rtm-docs/09-microservices/service-catalog.md`, `rtm-docs/07-api/contracts/openapi/`, and `rtm-docs/12-ux-ui/navigation-map.md`.

D.1 Sequence diagram — booking with bank transfer payment

It shows the communication among the API Gateway, `iam`, `booking-reservation`, `fleet-maintenance`, and `payment-billing` to create a booking and pay for it by bank transfer (uploading the receipt and its review by an administrator). Solid arrows represent synchronous REST calls; dashed arrows with an open tip represent asynchronous domain events.

Figure 29. *Sequence diagram of the booking with bank transfer payment.*

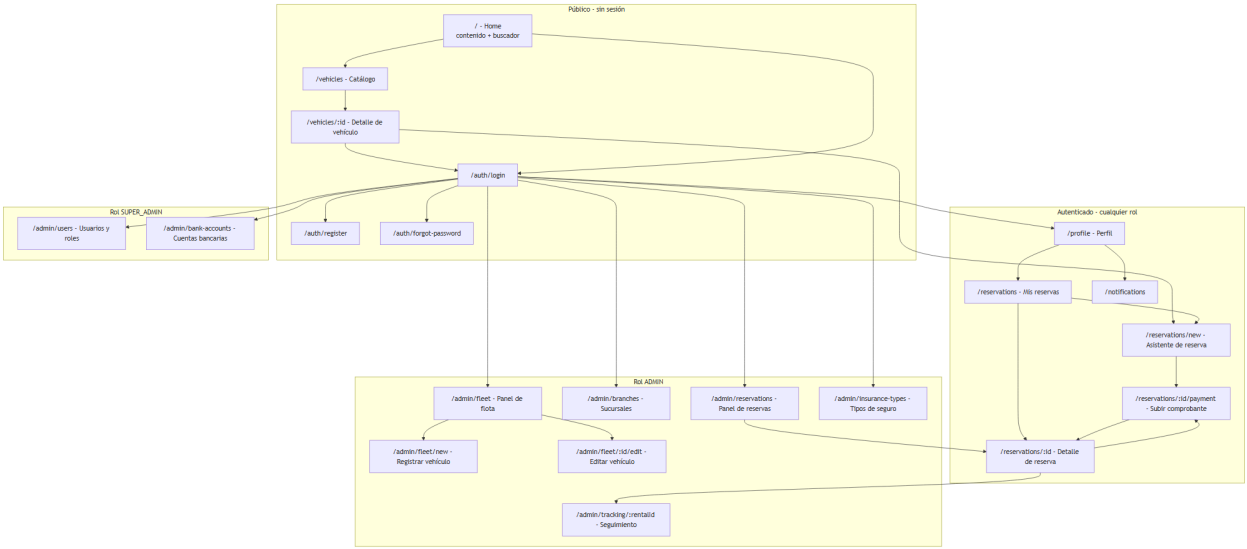


It corresponds to SEQ-03 "Create a booking" and SEQ-04 "Pay for a booking (receipt review)" in the analysis document. The endpoints and domain events were taken from the OpenAPI contracts and from `domain-events.md`.

D.2 Navigation map — 21 routes

Route structure of the web/mobile client and of the administrative interface, grouped by access level. The 21 screens described in the navigation map are listed.

Figure 30. *Navigation map of the 21 routes of the system.*



Source: `rtm-docs/12-ux-ui/navigation-map.md`. *The* `/admin/*` *routes require the ADMIN role;* `/admin/users` *and* `/admin/bank-accounts` *require SUPER_ADMIN. Booking requires a session: if there is none, the flow redirects to* `/auth/login`.

D.3 State diagram — booking and payment

Life cycles of `Reservation` and `Payment`.

Figure 31. State diagram of the booking (`Reservation.status`).

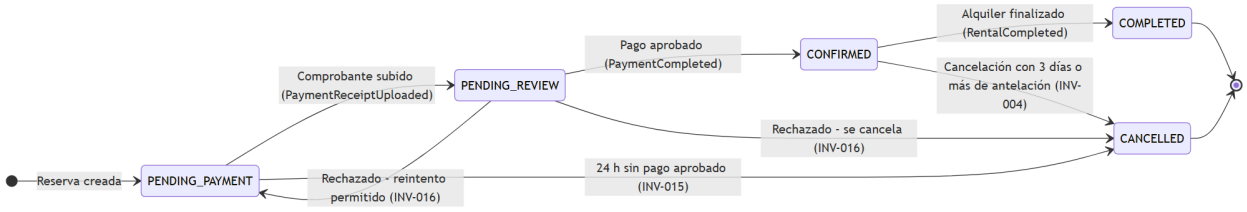
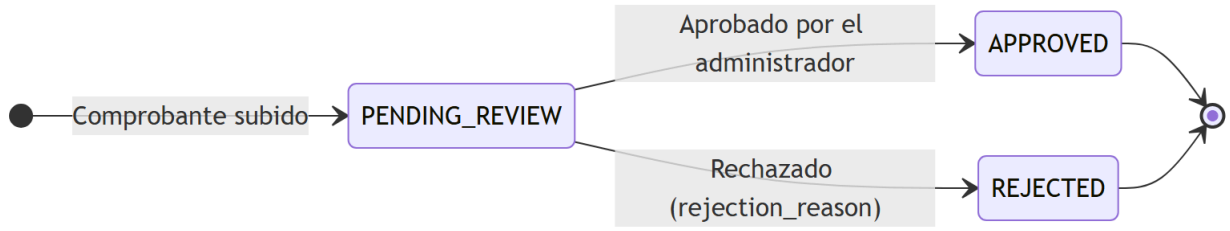


Figure 32. State diagram of the payment (`Payment.status`).



Source: `rtm-docs/02-domain/entities-and-rules.md` (INV-004, INV-015, INV-016) and `rtm-docs/02-domain/domain-events.md`. It corresponds to EST-01 "Reservation" and EST-02 "Payment" in the analysis document.